

Belhocine_Zied_1_notebook_022026

February 16, 2026

PROJET 4 DATA ANALYST

Réalisez une étude de santé publique avec R ou Python

1 OBJECTIF DE CE NOTEBOOK

Bienvenue dans l'outil plébiscité par les analystes de données Jupyter.

Il s'agit d'un outil permettant de mixer et d'alterner codes, textes et graphique.

Cet outil est formidable pour plusieurs raisons:

- il permet de tester des lignes de codes au fur et à mesure de votre rédaction, de constater immédiatement le résultat d'une instruction, de la corriger si nécessaire.
- De rédiger du texte pour expliquer l'approche suivie ou les résultats d'une analyse et de le mettre en forme grâce à du code html ou plus simple avec **Markdown**
- d'agrémenter de graphiques

Pour vous aider dans vos premiers pas à l'usage de Jupyter et de Python, nous avons rédigé ce notebook en vous indiquant les instructions à suivre.

Il vous suffit pour cela de saisir le code Python répondant à l'instruction donnée.

Vous verrez de temps à autre le code Python répondant à une instruction donnée mais cela est fait pour vous aider à comprendre la nature du travail qui vous est demandée.

Et garder à l'esprit, qu'il n'y a pas de solution unique pour résoudre un problème et qu'il y a autant de résolutions de problèmes que de développeurs ;)...

Etape 1 - Importation des librairies et chargement des fichiers

1.1 - Importation des librairies

```
[82]: #Importation de la librairie Pandas
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

1.2 - Chargement des fichiers Excel

```
[83]: #Importation du fichier population.csv
population = pd.read_csv('population.csv')
```

```
#Importation du fichier dispo_alimentaire.csv
dispo_alimentaire = pd.read_csv('dispo_alimentaire.csv')

#Importation du fichier aide_alimentaire.csv
aide_alimentaire = pd.read_csv('aide_alimentaire.csv')

#Importation du fichier sous_nutrition.csv
sous_nutrition = pd.read_csv('sous_nutrition.csv')
```

Etape 2 - Analyse exploratoire des fichiers

2.1 - Analyse exploratoire du fichier population

```
[84]: #Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(population.
↪shape[0]))
print("Le tableau comporte {} colonne(s)".format(population.shape[1]))
```

Le tableau comporte 1416 observation(s) ou article(s)

Le tableau comporte 3 colonne(s)

```
[85]: #Consulter le nombre de colonnes
print("Le tableau comporte {} colonne(s)".format(population.shape[1]))
#La nature des données dans chacune des colonnes
#Le nombre de valeurs présentes dans chacune des colonnes
population.info()
```

Le tableau comporte 3 colonne(s)

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1416 entries, 0 to 1415

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	Zone	1416 non-null	object
1	Année	1416 non-null	int64
2	Valeur	1416 non-null	float64

dtypes: float64(1), int64(1), object(1)

memory usage: 33.3+ KB

```
[86]: #Affichage les 5 premières lignes de la table
population.head()
```

```
[86]:
```

	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603
3	Afghanistan	2016	35383.032
4	Afghanistan	2017	36296.113

```
[87]: #Nous allons harmoniser les unités. Pour cela, nous avons décidé de multiplier
      ↪ la population par 1000
      #Multiplication de la colonne valeur par 1000
      population['Valeur'] = population['Valeur']*1000
```

```
[88]: #changement du nom de la colonne Valeur par Population
      population.rename(columns={'Valeur': 'Population'}, inplace=True)
```

```
[89]: #Affichage des 5 premières lignes de la table pour voir les modifications
      population.head()
```

```
[89]:
```

	Zone	Année	Population
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

2.2 - Analyse exploratoire du fichier disponibilité alimentaire

```
[90]: #Afficher les dimensions du dataset
      print(f"Le tableau comporte {dispo_alimentaire.shape[0]} observation(s) ou
      ↪ article(s) et {dispo_alimentaire.shape[1]} colonnes.")
```

Le tableau comporte 15605 observation(s) ou article(s) et 18 colonnes.

```
[91]: #Consulter le nombre de colonnes
      print("Le tableau comporte {} colonne(s)".format(dispo_alimentaire.shape[1]))
```

Le tableau comporte 18 colonne(s)

```
[92]: #Affichage des 5 premières lignes de la table
      dispo_alimentaire.head()
```

```
[92]:
```

	Zone	Produit	Origine	Aliments pour animaux	\
0	Afghanistan	Abats Comestible	animale		NaN
1	Afghanistan	Agrumes, Autres	vegetale		NaN
2	Afghanistan	Aliments pour enfants	vegetale		NaN
3	Afghanistan	Ananas	vegetale		NaN
4	Afghanistan	Bananes	vegetale		NaN

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
0	NaN		5.0
1	NaN		1.0
2	NaN		1.0
3	NaN		0.0
4	NaN		4.0

Disponibilité alimentaire en quantité (kg/personne/an) \

0	1.72
1	1.29
2	0.06
3	0.00
4	2.70

	Disponibilité de matière grasse en quantité (g/personne/jour) \
0	0.20
1	0.01
2	0.01
3	NaN
4	0.02

	Disponibilité de protéines en quantité (g/personne/jour) \
0	0.77
1	0.02
2	0.03
3	NaN
4	0.05

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité \
0	53.0	NaN	NaN
1	41.0	2.0	40.0
2	2.0	NaN	2.0
3	0.0	NaN	0.0
4	82.0	NaN	82.0

	Nourriture	Pertes	Production	Semences	Traitement	Variation de stock
0	53.0	NaN	53.0	NaN	NaN	NaN
1	39.0	2.0	3.0	NaN	NaN	NaN
2	2.0	NaN	NaN	NaN	NaN	NaN
3	0.0	NaN	NaN	NaN	NaN	NaN
4	82.0	NaN	NaN	NaN	NaN	NaN

```
[93]: #remplacement des NaN dans le dataset par des 0
dispo_alimentaire.fillna(0, inplace=True)
```

```
[94]: #multiplication de toutes les lignes contenant des milliers de tonnes en Kg
liste_colonnes_dispo = dispo_alimentaire.columns.tolist() # on crée une liste
↳avec tous les noms de colonnes
liste_non_modifie = ["Zone", "Produit", "Origine", "Disponibilité alimentaire",
↳(Kcal/personne/jour)", "Disponibilité alimentaire en quantité (kg/personne/
↳an)", "Disponibilité de matière grasse en quantité (g/personne/jour)",
↳"Disponibilité de protéines en quantité (g/personne/jour)"] # on crée une
↳liste avec tous les noms de colonnes qu'on ne souhaite pas modifier
```

```
colonnes_a_modifier = list(set(liste_colonnes_dispo) - set(liste_non_modifie))
↳# on fait la différence entre les 2 pour avoir uniquement la liste des
↳colonnes à modifier. On utilise l'opérateur '-' sur des sets de nos listes
↳car plus rapide à traiter en python qu'une boucle for.
```

```
[95]: colonnes_a_modifier
```

```
[95]: ['Disponibilité intérieure',
      'Importations - Quantité',
      'Semences',
      'Production',
      'Exportations - Quantité',
      'Aliments pour animaux',
      'Autres Utilisations',
      'Variation de stock',
      'Nourriture',
      'Traitement',
      'Pertes']
```

```
[96]: dispo_alimentaire[colonnes_a_modifier] *= (10**6) #conversion des colonnes
↳concernées par 1 000 000 (millier de tonnes = million de kg)
```

```
[97]: #Affichage les 5 premières lignes de la table
dispo_alimentaire.head()
```

```
[97]:
```

	Zone	Produit	Origine	Aliments pour animaux	\
0	Afghanistan	Abats Comestible	animale	0.0	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	
3	Afghanistan	Ananas	vegetale	0.0	
4	Afghanistan	Bananes	vegetale	0.0	

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
0	0.0	5.0	
1	0.0	1.0	
2	0.0	1.0	
3	0.0	0.0	
4	0.0	4.0	

	Disponibilité alimentaire en quantité (kg/personne/an)	\
0	1.72	
1	1.29	
2	0.06	
3	0.00	
4	2.70	

	Disponibilité de matière grasse en quantité (g/personne/jour)	\
--	---	---

0	0.20
1	0.01
2	0.01
3	0.00
4	0.02

	Disponibilité de protéines en quantité (g/personne/jour) \
0	0.77
1	0.02
2	0.03
3	0.00
4	0.05

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité \
0	53000000.0	0.0	0.0
1	41000000.0	2000000.0	40000000.0
2	2000000.0	0.0	2000000.0
3	0.0	0.0	0.0
4	82000000.0	0.0	82000000.0

	Nourriture	Pertes	Production	Semences	Traitement	Variation de stock
0	53000000.0	0.0	53000000.0	0.0	0.0	0.0
1	39000000.0	2000000.0	3000000.0	0.0	0.0	0.0
2	2000000.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0
4	82000000.0	0.0	0.0	0.0	0.0	0.0

2.3 - Analyse exploratoire du fichier aide alimentaire

```
[98]: #Afficher des dimensions du dataset
print(f"Le tableau comporte {aide_alimentaire.shape[0]} observation(s) ou
      ↪ article(s) et {aide_alimentaire.shape[1]} colonnes.")
```

Le tableau comporte 1475 observation(s) ou article(s) et 4 colonnes.

```
[99]: #Consulter le nombre de colonnes
print("Le tableau comporte {} colonne(s)".format(aide_alimentaire.shape[1]))
```

Le tableau comporte 4 colonne(s)

```
[100]: #Affichage les 5 premières lignes de la table
aide_alimentaire.head()
```

```
[100]:
```

	Pays bénéficiaire	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

```
[101]: #changement du nom de la colonne Pays bénéficiaire par Zone
aide_alimentaire.rename(columns={'Pays bénéficiaire': 'Zone'}, inplace=True)
```

```
[102]: #Multiplication de la colonne Aide_alimentaire qui contient des tonnes par 1000
        ↳pour avoir des kg
aide_alimentaire['Valeur'] *= 1000
```

```
[103]: #Affichage les 5 premières lignes de la table
aide_alimentaire.head()
```

```
[103]:
```

	Zone	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

2.3 - Analyse exploratoire du fichier sous nutrition

```
[104]: #Afficher les dimensions du dataset
print(f"Le tableau comporte {sous_nutrition.shape[0]} observation(s) ou
        ↳article(s) et {sous_nutrition.shape[1]} colonnes.")
```

Le tableau comporte 1218 observation(s) ou article(s) et 3 colonnes.

```
[105]: #Consulter le nombre de colonnes
print("Le tableau comporte {} colonne(s)".format(sous_nutrition.shape[1]))
```

Le tableau comporte 3 colonne(s).

```
[106]: #Afficher les 5 premières lignes de la table
sous_nutrition.head()
```

```
[106]:
```

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

```
[107]: #Conversion de la colonne sous nutrition en numérique
# sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur'])
# Il semble que la colonne comporte des valeurs qui ne sont pas des 'float'
        ↳('<0.1' par exemple ligne 60). Il faut donc utiliser l'argument
        ↳'errors=coerce' pour forcer le changement de type.
```

```
[108]: #Conversion de la colonne (avec l'argument errors=coerce qui permet de
        ↳convertir automatiquement les lignes qui ne sont pas des nombres en NaN)
```

```
sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur'],
↳errors='coerce')
```

```
[109]: #Puis remplacement des NaN en 0
sous_nutrition['Valeur'] = sous_nutrition['Valeur'].fillna(0)
```

```
[110]: #changement du nom de la colonne Valeur par sous_nutrition
sous_nutrition.rename(columns={'Valeur': 'sous_nutrition'}, inplace=True)
```

```
[111]: #Multiplication de la colonne sous_nutrition par 1000000
sous_nutrition['sous_nutrition'] *= (10**6)
```

```
[112]: #Afficher les 5 premières lignes de la table
sous_nutrition.head()
```

```
[112]:
```

	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8600000.0
1	Afghanistan	2013-2015	8800000.0
2	Afghanistan	2014-2016	8900000.0
3	Afghanistan	2015-2017	9700000.0
4	Afghanistan	2016-2018	10500000.0

3.1 - Proportion de personnes en sous nutrition

```
[113]: # Il faut tout d'abord faire une jointure entre la table population et la table
↳sous nutrition, en ciblant l'année 2017
# On va préparer le Dataframe sous_nutrition en modifiant la colonne année
print(sous_nutrition['Année'].unique())
# On s'assure qu'il n'y a que des intervalles de 3 ans dans le Dataframe
```

```
['2012-2014' '2013-2015' '2014-2016' '2015-2017' '2016-2018' '2017-2019']
```

```
[114]: sous_nutrition['Année'] = sous_nutrition['Année'].astype(str) #on transforme la
↳colonne en type str pour pouvoir la formater plus facilement.
sous_nutrition['Année'] = sous_nutrition['Année'].str.split('-',expand=True)[0].
↳astype(int) + 1 # expand renvoie un df sur lequel on peut appeler l'index de
↳la colonne : ici [0]
# On récupère la première année de chaque cellule de la colonne 'Année' à
↳laquelle on ajoute 1 pour obtenir l'année voulue (ex : 2016-2018 devient
↳2017 en int en plus !)
```

```
[115]: # On vérifie le dataset
sous_nutrition.head(7)
```

```
[115]:
```

	Zone	Année	sous_nutrition
0	Afghanistan	2013	8600000.0
1	Afghanistan	2014	8800000.0
2	Afghanistan	2015	8900000.0


```

3    Afghanistan    2016    9700000.0
4    Afghanistan    2017    10500000.0
5    Afghanistan    2018    11100000.0
6    Afrique du Sud    2013    2200000.0

```

```

[116]: # création d'un dataset population pour 2017
population_2017 = population.loc[population['Année'] == 2017]
# création d'un dataset sous_nutrition pour 2017
sous_nutrition_2017 = sous_nutrition.loc[sous_nutrition['Année'] == 2017]

```

```

[117]: # On vérifie que les deux colonnes 'Zone' ont bien les mêmes données : on fait
↳ la différence entre les 2 séries en les transformants en set car plus rapide.
manquants_sous_nutrition = list(set(population_2017['Zone']) -
↳ set(sous_nutrition_2017['Zone']))
manquants_population = list(set(sous_nutrition_2017['Zone']) -
↳ set(population_2017['Zone']))
print(f"Pays dans population mais pas dans sous_nutrition:
↳ {len(manquants_sous_nutrition)}")
print(f"Pays dans sous_nutrition mais pas dans population:
↳ {len(manquants_population)}")

```

Pays dans population mais pas dans sous_nutrition: 33

Pays dans sous_nutrition mais pas dans population: 0

```

[118]: # Il y a donc des pays dans 'population' qui ne se retrouvent pas dans
↳ 'sous_nutrition'. En revanche, on a tous les pays de 'sous-nutrition' qui
↳ sont avec la même orthographe dans 'population'.
# On va donc faire la jointure à gauche sur 'sous-nutrition' car tous les pays
↳ ont des correspondances dans l'autre Dataframe et on veut observer la
↳ sous_nutrition en premier lieu.

```

```

[119]: # jointure entre les deux datasets créés
proportion_sous_nutrition_2017 = pd.merge(sous_nutrition_2017, population_2017,
↳ on=['Zone', 'Année'], how="left")

```

```

[120]: # Remplacement des NaN par des 0
proportion_sous_nutrition_2017.fillna(0, inplace=True)

```

```

[121]: # Affichage du dataset
display(proportion_sous_nutrition_2017)

```

	Zone	Année	sous_nutrition	Population
0	Afghanistan	2017	10500000.0	36296113.0
1	Afrique du Sud	2017	3100000.0	57009756.0
2	Albanie	2017	100000.0	2884169.0
3	Algérie	2017	1300000.0	41389189.0
4	Allemagne	2017	0.0	82658409.0
..	...	...	...	...

198	Venezuela (République bolivarienne du)	2017	8000000.0	29402484.0
199	Viet Nam	2017	6500000.0	94600648.0
200	Yémen	2017	0.0	27834819.0
201	Zambie	2017	0.0	16853599.0
202	Zimbabwe	2017	0.0	14236595.0

[203 rows x 4 columns]

```
[122]: #Calcul et affichage du nombre de personnes en état de sous nutrition
print("Le nombre de personnes en sous nutrition est de {} millions.".
      ↪format(proportion_sous_nutrition_2017['sous_nutrition'].sum()/1000000)) #
      ↪Division par 10**6 pour afficher le nombre en millions et rendre cela plus
      ↪lisible.
print("Il y a {:.2f} milliards d'êtres humains sur terre.".
      ↪format(proportion_sous_nutrition_2017['Population'].sum()/10**9)) #idem ici
      ↪en milliards.
print("Cela représente une proportion de {:.2f}%.".
      ↪format(proportion_sous_nutrition_2017['sous_nutrition'].sum()*100/
      ↪proportion_sous_nutrition_2017['Population'].sum()))
```

Le nombre de personnes en sous nutrition est de 535.7 millions.

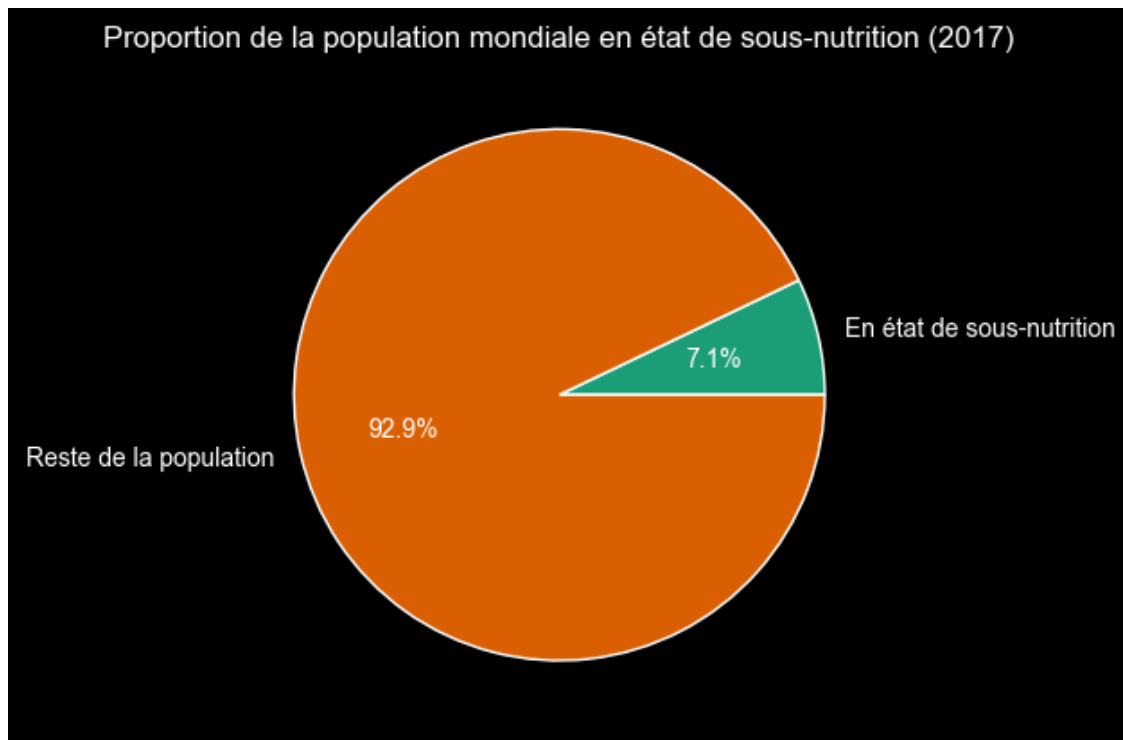
Il y a 7.54 milliards d'êtres humains sur terre.

Cela représente une proportion de 7.10%.

```
[123]: # On récupère les totaux déjà calculés
total_sous_nutrition = proportion_sous_nutrition_2017['sous_nutrition'].sum()
total_population = proportion_sous_nutrition_2017['Population'].sum()

# On calcule la part de la population "non en sous-nutrition"
reste_population = total_population - total_sous_nutrition

# On crée nos étiquettes et nos données
labels = ['En état de sous-nutrition', 'Reste de la population']
values = [total_sous_nutrition, reste_population]
colors = sns.color_palette('Dark2')
plt.pie(values, labels=labels, autopct='%1.1f%%', colors=colors)
plt.title("Proportion de la population mondiale en état de sous-nutrition,
      ↪(2017)")
plt.show()
```



3.2 - Nombre théorique de personne qui pourrait être nourries

```
[124]: #Combien mange en moyenne un être humain ?
nutrition_moyenne_par_jour = 2400
```

Combien mange en moyenne un être humain ? [Source](#) (en kcal/personne/j, moyenne mondiale)

```
[125]: dispo_alimentaire.head()
```

```
[125]:
```

	Zone	Produit	Origine	Aliments pour animaux	\
0	Afghanistan	Abats Comestible	animale	0.0	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	
3	Afghanistan	Ananas	vegetale	0.0	
4	Afghanistan	Bananes	vegetale	0.0	

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
0	0.0	5.0	
1	0.0	1.0	
2	0.0	1.0	
3	0.0	0.0	
4	0.0	4.0	

Disponibilité alimentaire en quantité (kg/personne/an) \

0	1.72
1	1.29
2	0.06
3	0.00
4	2.70

	Disponibilité de matière grasse en quantité (g/personne/jour) \
0	0.20
1	0.01
2	0.01
3	0.00
4	0.02

	Disponibilité de protéines en quantité (g/personne/jour) \
0	0.77
1	0.02
2	0.03
3	0.00
4	0.05

	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité \
0	53000000.0	0.0	0.0
1	41000000.0	2000000.0	40000000.0
2	2000000.0	0.0	2000000.0
3	0.0	0.0	0.0
4	82000000.0	0.0	82000000.0

	Nourriture	Pertes	Production	Semences	Traitement	Variation de stock
0	53000000.0	0.0	53000000.0	0.0	0.0	0.0
1	39000000.0	2000000.0	3000000.0	0.0	0.0	0.0
2	2000000.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0
4	82000000.0	0.0	0.0	0.0	0.0	0.0

```
[126]: # On vérifie que les deux colonnes 'Zone' ont bien les mêmes données : on fait
↳ la différence entre les 2 séries en les transformant en set car plus rapide.
manquants_dispo_alimentaire = list(set(population_2017['Zone'] ) -
↳ set(dispo_alimentaire['Zone']))
manquants_population = list(set(dispo_alimentaire['Zone'] ) -
↳ set(population_2017['Zone']))
print(f"Pays dans population mais pas dans sous_nutrition:
↳ {len(manquants_dispo_alimentaire)}")
print(f"Pays dans sous_nutrition mais pas dans population:
↳ {len(manquants_population)}")
```

Pays dans population mais pas dans sous_nutrition: 64
Pays dans sous_nutrition mais pas dans population: 2

```
[127]: # On observe les manquants de dispo_alimentaire
manquants_dispo_alimentaire
```

```
[127]: ['Samoa américaines',
        'Bonaire, Saint-Eustache et Saba',
        'Îles Vierges britanniques',
        'République démocratique du Congo',
        'Îles Wallis-et-Futuna',
        'Guadeloupe',
        'Martinique',
        'Tchéquie',
        'Qatar',
        'Saint-Marin',
        'Singapour',
        'Burundi',
        'Curaçao',
        'Sainte-Hélène, Ascension et Tristan da Cunha',
        'Érythrée',
        'Îles Turques-et-Caïques',
        'Tonga',
        'Libye',
        'Île de Man',
        'Palaos',
        'Îles Caïmanes',
        'Îles Cook',
        'Îles Mariannes du Nord',
        'Sint Maarten (partie néerlandaise)',
        'Soudan du Sud',
        'Saint-Pierre-et-Miquelon',
        'Porto Rico',
        'Mayotte',
        'Îles Vierges américaines',
        'Andorre',
        'Aruba',
        'Nioué',
        'Saint-Martin (partie française)',
        'Îles Féroé',
        'Monaco',
        'Anguilla',
        'Nauru',
        'Antilles néerlandaises (ex)',
        'Saint-Siège',
        'Somalie',
        'Gibraltar',
        'Îles Marshall',
        'Palestine',
        'Montserrat',
```

```
'Îles Falkland (Malvinas)',
'Tokélaou',
'Micronésie (États fédérés de)',
'Seychelles',
'Royaume-Uni de Grande-Bretagne et d'Irlande du Nord",
'Réunion',
'Papouasie-Nouvelle-Guinée',
'Guam',
'Îles Anglo-Normandes',
'Bhoutan',
'Guyane française',
'Bahreïn',
'Liechtenstein',
'Groenland',
'Tuvalu',
'République arabe syrienne',
'Guinée équatoriale',
'Comores',
'Saint-Barthélemy',
'Sahara occidental']
```

```
[128]: # On observe les manquants de population
manquants_population
```

```
[128]: ['Royaume-Uni', 'Tchéquie (la)']
```

```
[129]: # On renomme les lignes de population pour qu'elles correspondent avec celles
↳ de dispo_alimentaire
population_2017.loc[population_2017['Zone'] == "Royaume-Uni de Grande-Bretagne
↳ et d'Irlande du Nord", 'Zone'] = 'Royaume-Uni'
population_2017.loc[population_2017['Zone'] == "Tchéquie", 'Zone'] = 'Tchéquie
↳ (la)'
```

```
[130]: # On vérifie à nouveau
manquants_population = list(set(dispo_alimentaire['Zone'] ) -
↳ set(population_2017['Zone']))
print(f"Pays dans dispo_alimentaire mais pas dans population:
↳ {len(manquants_population)}")
```

Pays dans dispo_alimentaire mais pas dans population: 0

```
[131]: #Tous les pays sont de dispo_alimentaire ont bien une correspondance dans
↳ population.
# on peut maintenant ajouter la population au Dataframe dispo_alimentaire
dispo_alimentaire = pd.merge(dispo_alimentaire, population_2017, on='Zone',
↳ how="left")
```

```
[132]: #Affichage du nouveau dataframe
dispo_alimentaire
```

```
[132]:
```

	Zone	Produit	Origine	Aliments pour animaux	\
0	Afghanistan	Abats Comestible	animale	0.0	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	
3	Afghanistan	Ananas	vegetale	0.0	
4	Afghanistan	Bananes	vegetale	0.0	
...	...	...	...	...	
15600	Îles Salomon	Viande de Suides	animale	0.0	
15601	Îles Salomon	Viande de Volailles	animale	0.0	
15602	Îles Salomon	Viande, Autre	animale	0.0	
15603	Îles Salomon	Vin	vegetale	0.0	
15604	Îles Salomon	Épices, Autres	vegetale	0.0	

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
0	0.0	5.0	
1	0.0	1.0	
2	0.0	1.0	
3	0.0	0.0	
4	0.0	4.0	
...	...	...	
15600	0.0	45.0	
15601	0.0	11.0	
15602	0.0	0.0	
15603	0.0	0.0	
15604	0.0	4.0	

	Disponibilité alimentaire en quantité (kg/personne/an)	\
0	1.72	
1	1.29	
2	0.06	
3	0.00	
4	2.70	
...	...	
15600	4.70	
15601	3.34	
15602	0.06	
15603	0.07	
15604	0.48	

	Disponibilité de matière grasse en quantité (g/personne/jour)	\
0	0.20	
1	0.01	
2	0.01	
3	0.00	

4	0.02
...	...
15600	4.28
15601	0.69
15602	0.00
15603	0.00
15604	0.21

Disponibilité de protéines en quantité (g/personne/jour) \	
0	0.77
1	0.02
2	0.03
3	0.00
4	0.05
...	...
15600	1.41
15601	1.14
15602	0.04
15603	0.00
15604	0.15

Disponibilité intérieure Exportations - Quantité \		
0	53000000.0	0.0
1	41000000.0	2000000.0
2	2000000.0	0.0
3	0.0	0.0
4	82000000.0	0.0
...	...	...
15600	3000000.0	0.0
15601	2000000.0	0.0
15602	0.0	0.0
15603	0.0	0.0
15604	0.0	0.0

Importations - Quantité	Nourriture	Pertes	Production	Semences \	
0	0.0	53000000.0	0.0	53000000.0	0.0
1	40000000.0	39000000.0	2000000.0	3000000.0	0.0
2	2000000.0	2000000.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0
4	82000000.0	82000000.0	0.0	0.0	0.0
...	...	...	...	...	...
15600	0.0	3000000.0	0.0	2000000.0	0.0
15601	2000000.0	2000000.0	0.0	0.0	0.0
15602	0.0	0.0	0.0	0.0	0.0
15603	0.0	0.0	0.0	0.0	0.0
15604	0.0	0.0	0.0	0.0	0.0

	Traitement	Variation de stock	Année	Population
0	0.0	0.0	2017	36296113.0
1	0.0	0.0	2017	36296113.0
2	0.0	0.0	2017	36296113.0
3	0.0	0.0	2017	36296113.0
4	0.0	0.0	2017	36296113.0
...	...	...	...	...
15600	0.0	0.0	2017	636039.0
15601	0.0	0.0	2017	636039.0
15602	0.0	0.0	2017	636039.0
15603	0.0	0.0	2017	636039.0
15604	0.0	0.0	2017	636039.0

[15605 rows x 20 columns]

```
[133]: #Création de la colonne dispo_kcal avec calcul des kcal disponibles
↳mondialement en multipliant la dispo kcal/pers/j par la population.
dispo_alimentaire['dispo_kcal par jour'] = dispo_alimentaire['Disponibilité_
↳alimentaire (Kcal/personne/jour)'] * dispo_alimentaire['Population']
```

```
[134]: dispo_alimentaire
```

```
[134]:
```

	Zone	Produit	Origine	Aliments pour animaux \
0	Afghanistan	Abats Comestible	animale	0.0
1	Afghanistan	Agrumes, Autres	vegetale	0.0
2	Afghanistan	Aliments pour enfants	vegetale	0.0
3	Afghanistan	Ananas	vegetale	0.0
4	Afghanistan	Bananes	vegetale	0.0
...	...	...	...	...
15600	Îles Salomon	Viande de Suides	animale	0.0
15601	Îles Salomon	Viande de Volailles	animale	0.0
15602	Îles Salomon	Viande, Autre	animale	0.0
15603	Îles Salomon	Vin	vegetale	0.0
15604	Îles Salomon	Épices, Autres	vegetale	0.0

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour) \
0	0.0	5.0
1	0.0	1.0
2	0.0	1.0
3	0.0	0.0
4	0.0	4.0
...	...	...
15600	0.0	45.0
15601	0.0	11.0
15602	0.0	0.0
15603	0.0	0.0
15604	0.0	4.0

	Disponibilité alimentaire en quantité (kg/personne/an) \
0	1.72
1	1.29
2	0.06
3	0.00
4	2.70
...	...
15600	4.70
15601	3.34
15602	0.06
15603	0.07
15604	0.48

	Disponibilité de matière grasse en quantité (g/personne/jour) \
0	0.20
1	0.01
2	0.01
3	0.00
4	0.02
...	...
15600	4.28
15601	0.69
15602	0.00
15603	0.00
15604	0.21

	Disponibilité de protéines en quantité (g/personne/jour) \
0	0.77
1	0.02
2	0.03
3	0.00
4	0.05
...	...
15600	1.41
15601	1.14
15602	0.04
15603	0.00
15604	0.15

	Disponibilité intérieure	...	Importations - Quantité	Nourriture \
0	53000000.0	...	0.0	53000000.0
1	41000000.0	...	40000000.0	39000000.0
2	2000000.0	...	2000000.0	2000000.0
3	0.0	...	0.0	0.0
4	82000000.0	...	82000000.0	82000000.0
...	...	...	...	...

15600	3000000.0	...	0.0	3000000.0
15601	2000000.0	...	2000000.0	2000000.0
15602	0.0	...	0.0	0.0
15603	0.0	...	0.0	0.0
15604	0.0	...	0.0	0.0

	Pertes	Production	Semences	Traitement	Variation de stock	Année \
0	0.0	53000000.0	0.0	0.0	0.0	2017
1	2000000.0	3000000.0	0.0	0.0	0.0	2017
2	0.0	0.0	0.0	0.0	0.0	2017
3	0.0	0.0	0.0	0.0	0.0	2017
4	0.0	0.0	0.0	0.0	0.0	2017
...	...	...	...	...	...	...
15600	0.0	2000000.0	0.0	0.0	0.0	2017
15601	0.0	0.0	0.0	0.0	0.0	2017
15602	0.0	0.0	0.0	0.0	0.0	2017
15603	0.0	0.0	0.0	0.0	0.0	2017
15604	0.0	0.0	0.0	0.0	0.0	2017

	Population	dispo_kcal par jour
0	36296113.0	181480565.0
1	36296113.0	36296113.0
2	36296113.0	36296113.0
3	36296113.0	0.0
4	36296113.0	145184452.0
...	...	...
15600	636039.0	28621755.0
15601	636039.0	6996429.0
15602	636039.0	0.0
15603	636039.0	0.0
15604	636039.0	2544156.0

[15605 rows x 21 columns]

```
[135]: #Calcul du nombre d'humains pouvant être nourris
print("Il y a {:.2f} milliards d'êtres humains sur terre.".
      ↪format(population_2017['Population'].sum()/10**9))
total_humains_nourris = round(dispo_alimentaire['dispo_kcal par jour'].sum() / ↪
      ↪nutrition_moyenne_par_jour)
print("Le nombre d'humains pouvant être nourris est de {:.2f} milliards.".
      ↪format(total_humains_nourris/10**9))
print("On peut donc nourrir {:.2f} % de la population mondiale.".
      ↪format(100*total_humains_nourris/population_2017['Population'].sum()))
```

Il y a 7.55 milliards d'êtres humains sur terre.
 Le nombre d'humains pouvant être nourris est de 8.83 milliards.
 On peut donc nourrir 116.93 % de la population mondiale.

3.3 - Nombre théorique de personne qui pourrait être nourrie avec les produits végétaux

```
[136]: #Transfert des données avec les végétaux dans un nouveau dataframe
dispo_alimentaire_vegetaux = dispo_alimentaire.loc[dispo_alimentaire['Origine']_
↳ == 'vegetale']
```

```
[137]: dispo_alimentaire_vegetaux
```

```
[137]:
```

	Zone	Produit	Origine	Aliments pour animaux	\
1	Afghanistan	Agrumes, Autres	vegetale	0.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	
3	Afghanistan	Ananas	vegetale	0.0	
4	Afghanistan	Bananes	vegetale	0.0	
6	Afghanistan	Bière	vegetale	0.0	
...	...	...	...	...	
15595	Îles Salomon	Sésame	vegetale	0.0	
15596	Îles Salomon	Thé	vegetale	0.0	
15597	Îles Salomon	Tomates	vegetale	0.0	
15603	Îles Salomon	Vin	vegetale	0.0	
15604	Îles Salomon	Épices, Autres	vegetale	0.0	

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
1	0.0	1.0	
2	0.0	1.0	
3	0.0	0.0	
4	0.0	4.0	
6	0.0	0.0	
...	...	...	
15595	0.0	0.0	
15596	0.0	0.0	
15597	0.0	0.0	
15603	0.0	0.0	
15604	0.0	4.0	

	Disponibilité alimentaire en quantité (kg/personne/an)	\
1	1.29	
2	0.06	
3	0.00	
4	2.70	
6	0.09	
...	...	
15595	0.01	
15596	0.17	
15597	0.07	
15603	0.07	
15604	0.48	

	Disponibilité de matière grasse en quantité (g/personne/jour) \
1	0.01
2	0.01
3	0.00
4	0.02
6	0.00
...	...
15595	0.00
15596	0.00
15597	0.00
15603	0.00
15604	0.21

	Disponibilité de protéines en quantité (g/personne/jour) \
1	0.02
2	0.03
3	0.00
4	0.05
6	0.00
...	...
15595	0.00
15596	0.04
15597	0.00
15603	0.00
15604	0.15

	Disponibilité intérieure	...	Importations - Quantité	Nourriture \
1	41000000.0	...	40000000.0	39000000.0
2	2000000.0	...	2000000.0	2000000.0
3	0.0	...	0.0	0.0
4	82000000.0	...	82000000.0	82000000.0
6	3000000.0	...	3000000.0	3000000.0
...	...	...	...	...
15595	0.0	...	0.0	0.0
15596	0.0	...	0.0	0.0
15597	0.0	...	0.0	0.0
15603	0.0	...	0.0	0.0
15604	0.0	...	0.0	0.0

	Pertes	Production	Semences	Traitement	Variation de stock	Année \
1	2000000.0	3000000.0	0.0	0.0	0.0	2017
2	0.0	0.0	0.0	0.0	0.0	2017
3	0.0	0.0	0.0	0.0	0.0	2017
4	0.0	0.0	0.0	0.0	0.0	2017
6	0.0	0.0	0.0	0.0	0.0	2017
...	...	...	...	...	...	...
15595	0.0	0.0	0.0	0.0	0.0	2017

15596	0.0	0.0	0.0	0.0	0.0	2017
15597	0.0	0.0	0.0	0.0	0.0	2017
15603	0.0	0.0	0.0	0.0	0.0	2017
15604	0.0	0.0	0.0	0.0	0.0	2017

	Population	dispo_kcal par jour
1	36296113.0	36296113.0
2	36296113.0	36296113.0
3	36296113.0	0.0
4	36296113.0	145184452.0
6	36296113.0	0.0
...	...	...
15595	636039.0	0.0
15596	636039.0	0.0
15597	636039.0	0.0
15603	636039.0	0.0
15604	636039.0	2544156.0

[11896 rows x 21 columns]

```
[138]: #Calcul du nombre de kcal disponible pour les végétaux
kcal_dispo_vegetaux_par_jour = dispo_alimentaire_vegetaux['dispo_kcal par_
    jour'].sum()
print("Le nombre de kcal disponible par jour pour les végétaux est de {:.2f}_
    milliards.".format(kcal_dispo_vegetaux_par_jour/10**9))
```

Le nombre de kcal disponible par jour pour les végétaux est de 17449.51 milliards.

```
[139]: #Calcul du nombre d'humains pouvant être nourris avec les végétaux
total_humains_nourris_vegetaux = round(dispo_alimentaire_vegetaux['dispo_kcal_
    par jour'].sum() / nutrition_moyenne_par_jour)
print("Le nombre d'humains pouvant être nourris avec les végétaux est de {:.2f}_
    milliards.".format(total_humains_nourris_vegetaux/10**9))
print("On peut donc nourrir {:.2f} % de la population mondiale avec des_
    végétaux.".format(100*total_humains_nourris_vegetaux/
    population_2017['Population'].sum()))
```

Le nombre d'humains pouvant être nourris avec les végétaux est de 7.27 milliards.

On peut donc nourrir 96.32 % de la population mondiale avec des végétaux.

3.4 - Utilisation de la disponibilité intérieure

```
[140]: #Calcul de la disponibilité totale
dispo_totale = dispo_alimentaire['Disponibilité intérieure'].sum()
print("La disponibilité totale est de {} milliards de kgs par jour.".
    format(dispo_totale/(10**9)))
```

La disponibilité totale est de 9848.994 milliards de kgs par jour.

```
[141]: #création d'une boucle for pour afficher les différentes valeurs en fonction
        ↪des colonnes aliments pour animaux, pertes, nourritures,
liste_boucle = ['Aliments pour animaux', 'Pertes', 'Nourriture']

for elt in liste_boucle:
    (print(f"La Dispo de {elt} est de {dispo_alimentaire[elt].sum()/(10**9):.
        ↪1f} milliards kg, cela représente {dispo_alimentaire[elt].sum()*100/
        ↪dispo_totale:.2f}% de la dispo totale."))
```

La Dispo de Aliments pour animaux est de 1304.2 milliards kg, cela représente 13.24% de la dispo totale.

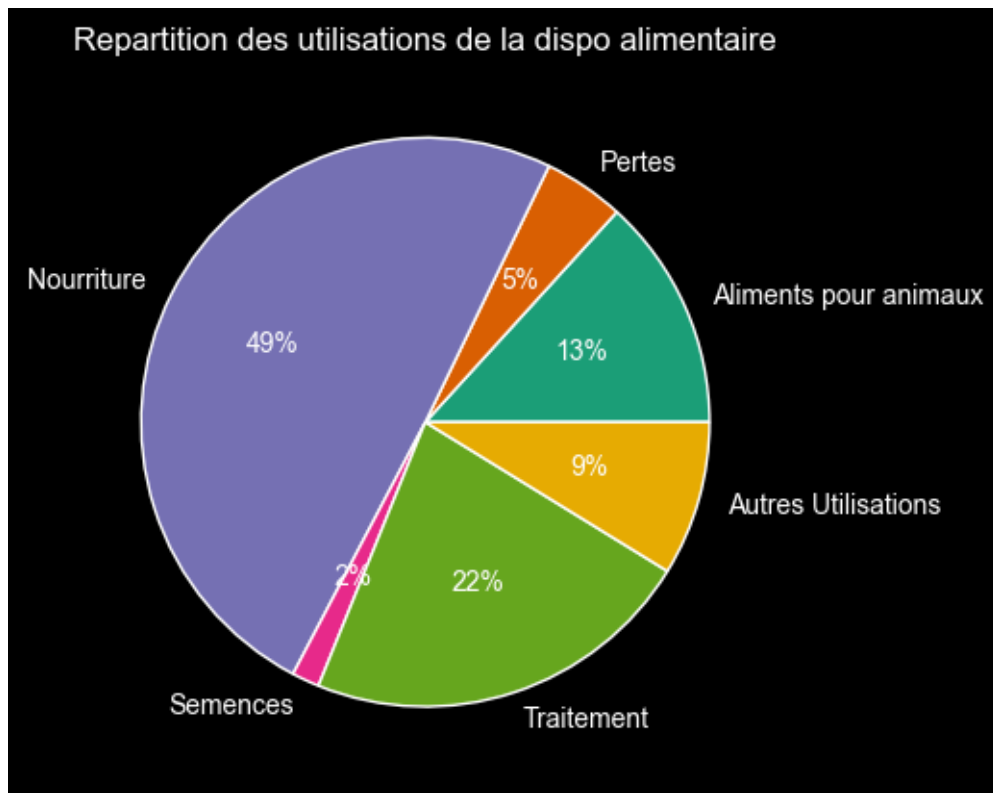
La Dispo de Pertes est de 453.7 milliards kg, cela représente 4.61% de la dispo totale.

La Dispo de Nourriture est de 4876.3 milliards kg, cela représente 49.51% de la dispo totale.

```
[142]: # si on veut observer toutes les utilisations on peut créer une liste qui va
        ↪contenir la somme totale pour chaque catégorie
liste_boucle_utilisation_dispo = ['Aliments pour animaux', 'Pertes',
        ↪'Nourriture', 'Semences', 'Traitement', 'Autres Utilisations'] # Ici on a
        ↪une liste de chaque catégorie sur laquelle on va itérer
categories_utilisation_dispo = [] # Ici on a une liste vide qui va accueillir
        ↪les totaux de chaque catégorie

for elt in liste_boucle_utilisation_dispo: # la boucle for pour ajouter les
        ↪totaux à notre liste vide
    categories_utilisation_dispo.append(dispo_alimentaire[elt].sum())

# création du graphique
colors = sns.color_palette('Dark2')
plt.title("Repartition des utilisations de la dispo alimentaire")
plt.pie(categories_utilisation_dispo, labels=liste_boucle_utilisation_dispo,
        ↪colors=colors, autopct='%.0f%%')
plt.show()
```



```
[143]: #On peut faire pareil pour la provenance de la dispo
liste_boucle_provenance_dispo = ['Production', 'Importations - Quantité',
    ↪ 'Exportations - Quantité', 'Variation de stock'] # Ici, on a une liste de
    ↪ chaque catégorie sur laquelle on va itérer
df_total_balance_provenance_dispo =
    ↪ dispo_alimentaire[list_boucle_provenance_dispo].sum().reset_index() #
    ↪ création du df avec les sommes de chaque catégorie
```

```
[144]: df_total_balance_provenance_dispo
```

```
[144]:
```

	index	0
0	Production	1.000968e+13
1	Importations - Quantité	1.296053e+12
2	Exportations - Quantité	1.352158e+12
3	Variation de stock	-1.044020e+11

```
[145]: #C'est bien mais il faut encore travailler sur le DF pour lui ajouter les
    ↪ informations que l'on souhaite
df_total_balance_provenance_dispo.columns = ['Catégorie', 'Valeur_totale en
    ↪ milliards de kgs'] # On nomme nos colonnes
df_total_balance_provenance_dispo['Valeur_totale en milliards de kgs'] /= 10**9
    ↪ # Conversion en milliard pour plus de lisibilité
```



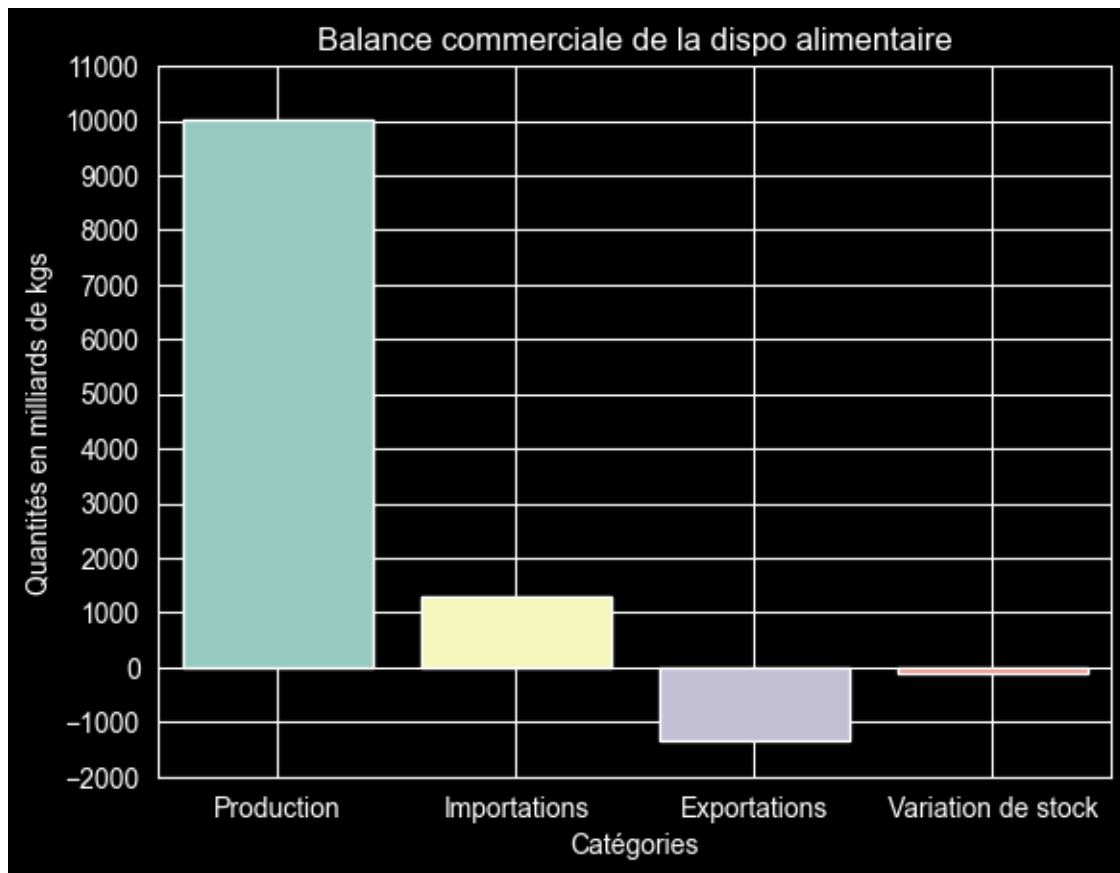
```
[146]: df_total_balance_provenance_dispo['Valeur_totale en milliards de kgs'] = pd.
        ↪to_numeric(df_total_balance_provenance_dispo['Valeur_totale en milliards de_
        ↪kgs'], errors='coerce') # Conversion en float pour pouvoir faire un * -1 sur_
        ↪les exportations et qu'elles soient représentées comme négatives
df_total_balance_provenance_dispo.at[2, 'Valeur_totale en milliards de kgs'] *=_
        ↪-1 # l'argument "2" fait référence à la ligne exportations
df_total_balance_provenance_dispo#vérification de nos opérations
```

```
[146]:
```

	Catégorie	Valeur_totale en milliards de kgs
0	Production	10009.680
1	Importations - Quantité	1296.053
2	Exportations - Quantité	-1352.158
3	Variation de stock	-104.402

```
[147]: # On renomme nos colonnes pour plus de lisibilité
df_total_balance_provenance_dispo.
        ↪loc[df_total_balance_provenance_dispo['Catégorie'] == 'Exportations -_
        ↪Quantité', 'Catégorie'] = 'Exportations'
df_total_balance_provenance_dispo.
        ↪loc[df_total_balance_provenance_dispo['Catégorie'] == 'Importations -_
        ↪Quantité', 'Catégorie'] = 'Importations'
```

```
[202]: # création du graphique en barres
sns.barplot(x=df_total_balance_provenance_dispo['Catégorie'],_
        ↪y=df_total_balance_provenance_dispo['Valeur_totale en milliards de kgs'],_
        ↪hue=df_total_balance_provenance_dispo['Catégorie'])
plt.title("Balance commerciale de la dispo alimentaire")
plt.xlabel("Catégories")
plt.ylabel("Quantités en milliards de kgs")
plt.yticks(range(-2000, 12000, 1000))
plt.grid(axis='x')
plt.show()
```



3.5 - Utilisation des céréales

```
[149]: #Création d'une liste avec toutes les variables
cereales = ['Avoine', 'Blé', 'Céréales, Autres', 'Maïs', 'Millet', 'Orge', 'Riz',
            '(Eq Blanchi)', 'Seigle', 'Sorgho']
```

```
[150]: #Création d'un dataframe avec les informations uniquement pour ces céréales
masque_cereales = dispo_alimentaire['Produit'].isin(cereales)
dispo_cereales = dispo_alimentaire.loc[masque_cereales]
dispo_cereales
```

```
[150]:
```

	Zone	Produit	Origine	Aliments pour animaux \
7	Afghanistan	Blé	vegetale	0.0
12	Afghanistan	Céréales, Autres	vegetale	0.0
32	Afghanistan	Maïs	vegetale	200000000.0
34	Afghanistan	Millet	vegetale	0.0
40	Afghanistan	Orge	vegetale	360000000.0
...	...	...	...	...
15545	Îles Salomon	Céréales, Autres	vegetale	0.0
15568	Îles Salomon	Maïs	vegetale	0.0

15575	Îles Salomon	Orge	vegetale	0.0
15591	Îles Salomon	Riz (Eq Blanchi)	vegetale	0.0
15593	Îles Salomon	Sorgho	vegetale	0.0

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	\
7	0.0	1369.0	
12	0.0	0.0	
32	0.0	21.0	
34	0.0	3.0	
40	0.0	26.0	
...	...	...	
15545	0.0	0.0	
15568	0.0	1.0	
15575	0.0	0.0	
15591	12000000.0	623.0	
15593	0.0	0.0	

	Disponibilité alimentaire en quantité (kg/personne/an)	\
7	160.23	
12	0.00	
32	2.50	
34	0.40	
40	2.92	
...	...	
15545	0.00	
15568	0.15	
15575	0.07	
15591	63.76	
15593	0.00	

	Disponibilité de matière grasse en quantité (g/personne/jour)	\
7	4.69	
12	0.00	
32	0.30	
34	0.02	
40	0.24	
...	...	
15545	0.00	
15568	0.01	
15575	0.00	
15591	1.36	
15593	0.00	

	Disponibilité de protéines en quantité (g/personne/jour)	\
7	36.91	
12	0.00	
32	0.56	

34	0.08
40	0.79
...	...
15545	0.00
15568	0.03
15575	0.01
15591	10.90
15593	0.00

	Disponibilité intérieure	...	Importations - Quantité	Nourriture \
7	5.992000e+09	...	1.173000e+09	4.895000e+09
12	0.000000e+00	...	0.000000e+00	0.000000e+00
32	3.130000e+08	...	1.000000e+06	7.600000e+07
34	1.300000e+07	...	0.000000e+00	1.200000e+07
40	5.240000e+08	...	1.000000e+07	8.900000e+07
...	...	...	...	...
15545	0.000000e+00	...	0.000000e+00	0.000000e+00
15568	0.000000e+00	...	0.000000e+00	0.000000e+00
15575	1.000000e+06	...	1.000000e+06	0.000000e+00
15591	4.900000e+07	...	4.700000e+07	3.600000e+07
15593	0.000000e+00	...	0.000000e+00	0.000000e+00

	Pertes	Production	Semences	Traitement	Variation de stock \
7	775000000.0	5.169000e+09	322000000.0	0.0	-350000000.0
12	0.0	0.000000e+00	0.0	0.0	0.0
32	31000000.0	3.120000e+08	5000000.0	0.0	0.0
34	1000000.0	1.300000e+07	0.0	0.0	0.0
40	52000000.0	5.140000e+08	22000000.0	0.0	0.0
...	...	...	...	...	...
15545	0.0	0.000000e+00	0.0	0.0	0.0
15568	0.0	0.000000e+00	0.0	0.0	0.0
15575	0.0	0.000000e+00	0.0	1000000.0	0.0
15591	1000000.0	3.000000e+06	0.0	0.0	0.0
15593	0.0	0.000000e+00	0.0	0.0	0.0

	Année	Population	dispo_kcal par jour
7	2017	36296113.0	4.968938e+10
12	2017	36296113.0	0.000000e+00
32	2017	36296113.0	7.622184e+08
34	2017	36296113.0	1.088883e+08
40	2017	36296113.0	9.436989e+08
...	...	...	...
15545	2017	636039.0	0.000000e+00
15568	2017	636039.0	6.360390e+05
15575	2017	636039.0	0.000000e+00
15591	2017	636039.0	3.962523e+08
15593	2017	636039.0	0.000000e+00

[1497 rows x 21 columns]

```
[151]: #Affichage de la proportion d'alimentation animale
proportion_alim_animale = dispo_cereales['Aliments pour animaux'].sum()*100 /
↳dispo_cereales['Disponibilité intérieure'].sum()
print("La proportion d'alimentation animale dans l'utilisation des céréales est,
↳de {:.2f}%".format(proportion_alim_animale))
```

La proportion d'alimentation animale dans l'utilisation des céréales est de 36.29%.

```
[152]: #création d'un dataframe avec les totaux par céréales où on ne garde que les
↳colonnes qui nous intéressent.
dispo_cereales_somme = dispo_cereales.groupby('Produit')[['Aliments pour
↳animaux', 'Nourriture', 'Disponibilité intérieure']].sum().reset_index()
total_animaux = dispo_cereales_somme['Aliments pour animaux'].sum()
total_nourriture = dispo_cereales_somme['Nourriture'].sum()
total_dispo = dispo_cereales_somme['Disponibilité intérieure'].sum()
```

```
[153]: dispo_cereales_somme
```

```
[153]:
```

	Produit	Aliments pour animaux	Nourriture \
0	Avoine	1.625100e+10	3.903000e+09
1	Blé	1.296680e+11	4.578240e+11
2	Céréales, Autres	1.903500e+10	5.324000e+09
3	Maïs	5.461160e+11	1.251840e+11
4	Millet	3.306000e+09	2.304000e+10
5	Orge	9.265800e+10	6.794000e+09
6	Riz (Eq Blanchi)	3.359400e+10	3.772860e+11
7	Seigle	8.099000e+09	5.502000e+09
8	Sorgho	2.480800e+10	2.415300e+10

	Disponibilité intérieure
0	2.340700e+10
1	6.794980e+11
2	2.748500e+10
3	9.557990e+11
4	2.991100e+10
5	1.404390e+11
6	4.756560e+11
7	1.656700e+10
8	5.823700e+10

```
[154]: #création d'un DF pour avoir une ligne total pour l'ensemble des céréales
↳qu'on va par la suite concaténer à notre DF dispo_cereales_somme.
```

```

# On commence par créer nos variables qui vont être utilisées dans notre futur
↪DF
total_animaux = dispo_cereales_somme['Aliments pour animaux'].sum()
total_nourriture = dispo_cereales_somme['Nourriture'].sum()
total_dispo = dispo_cereales_somme['Disponibilité intérieure'].sum()

#Création du DF
df_categorie_total_cereales = pd.DataFrame({'Produit': ['Total_
↪Céréales'], 'Aliments pour animaux': [total_animaux], 'Nourriture':
↪[total_nourriture], 'Disponibilité intérieure': [total_dispo]})

```

```
[155]: df_categorie_total_cereales
```

```

[155]:          Produit  Aliments pour animaux  Nourriture \
0  Total Céréales      8.735350e+11  1.029010e+12

      Disponibilité intérieure
0                2.406999e+12

```

```

[156]: # on a bien le même nombre de colonnes, on peut donc concaténer les 2 DFs.
dispo_cereales_somme = pd.concat([dispo_cereales_somme,
↪df_categorie_total_cereales])
#Ajout de deux colonnes pour avoir la proportion pour les deux catégories
dispo_cereales_somme['proportion aliments pour animaux'] = 100 *
↪dispo_cereales_somme['Aliments pour animaux']/
↪dispo_cereales_somme['Disponibilité intérieure']
dispo_cereales_somme['proportion alimentation humaine'] = 100 *
↪dispo_cereales_somme['Nourriture']/ dispo_cereales_somme['Disponibilité_
↪intérieure']

```

```
[157]: dispo_cereales_somme
```

```

[157]:          Produit  Aliments pour animaux  Nourriture \
0          Avoine      1.625100e+10  3.903000e+09
1           Blé      1.296680e+11  4.578240e+11
2  Céréales, Autres      1.903500e+10  5.324000e+09
3           Maïs      5.461160e+11  1.251840e+11
4          Millet      3.306000e+09  2.304000e+10
5           Orge      9.265800e+10  6.794000e+09
6  Riz (Eq Blanchi)      3.359400e+10  3.772860e+11
7           Seigle      8.099000e+09  5.502000e+09
8           Sorgho      2.480800e+10  2.415300e+10
0    Total Céréales      8.735350e+11  1.029010e+12

      Disponibilité intérieure  proportion aliments pour animaux \
0          2.340700e+10                69.427949
1          6.794980e+11                19.082911

```

2	2.748500e+10	69.255958
3	9.557990e+11	57.137118
4	2.991100e+10	11.052790
5	1.404390e+11	65.977399
6	4.756560e+11	7.062667
7	1.656700e+10	48.886340
8	5.823700e+10	42.598348
0	2.406999e+12	36.291457

proportion alimentation humaine	
0	16.674499
1	67.376799
2	19.370566
3	13.097314
4	77.028518
5	4.837688
6	79.319088
7	33.210599
8	41.473634
0	42.750745

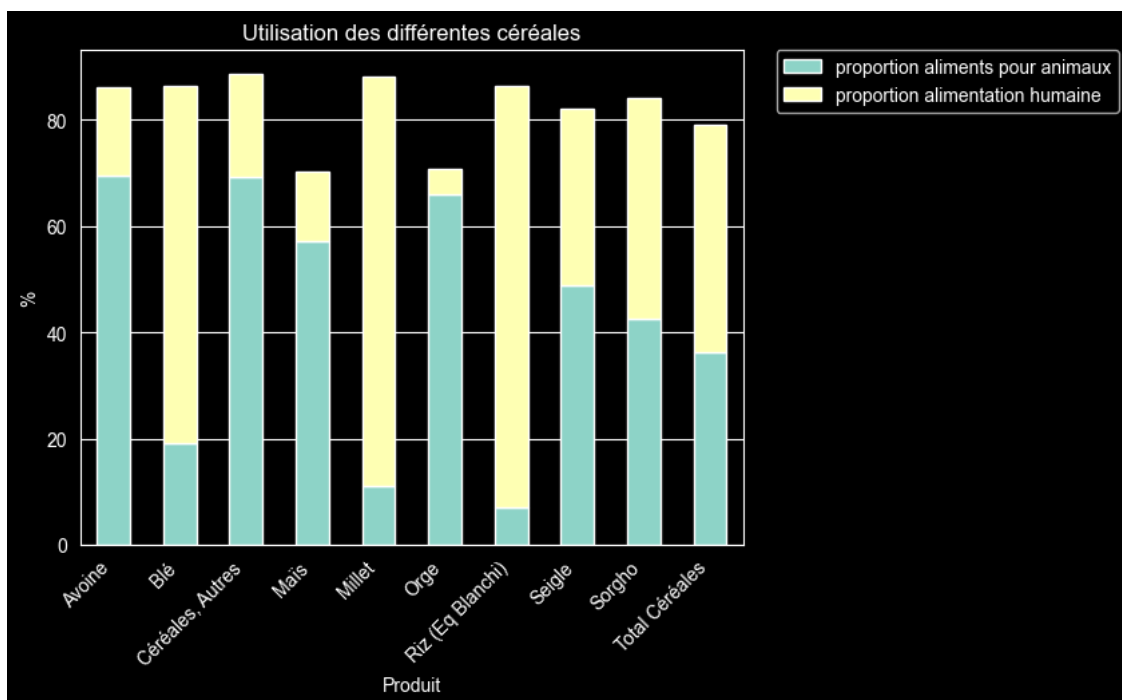
```
[158]: dispo_cereales_somme.set_index('Produit')[['proportion aliments pour animaux',
↳ 'proportion alimentation humaine']]
```

```
[158]: proportion aliments pour animaux \
Produit
Avoine                69.427949
Blé                   19.082911
Céréales, Autres     69.255958
Maïs                  57.137118
Millet               11.052790
Orge                  65.977399
Riz (Eq Blanchi)      7.062667
Seigle                48.886340
Sorgho                42.598348
Total Céréales       36.291457
```

proportion alimentation humaine	
Produit	
Avoine	16.674499
Blé	67.376799
Céréales, Autres	19.370566
Maïs	13.097314
Millet	77.028518
Orge	4.837688
Riz (Eq Blanchi)	79.319088
Seigle	33.210599

Sorgho	41.473634
Total Céréales	42.750745

```
[159]: # Création d'un barchart pour représenter les proportions par catégorie de
        ↪céréale.
dispo_cereales_somme.set_index('Produit')[['proportion aliments pour animaux',
        ↪'proportion alimentation humaine']].plot(kind='bar', stacked=True)
plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0.)
plt.title("Utilisation des différentes céréales")
plt.grid(axis='x')
plt.xticks(rotation=45, ha='right')
plt.ylabel('%')
plt.show()
```



Annexe : 2ème façon de faire le graphique

```
[160]: dispo_cereales_somme_2 = dispo_cereales_somme.melt(id_vars='Produit',
        ↪value_vars=['proportion aliments pour animaux', 'proportion alimentation
        ↪humaine'], var_name='Type', value_name='Proportion')
```

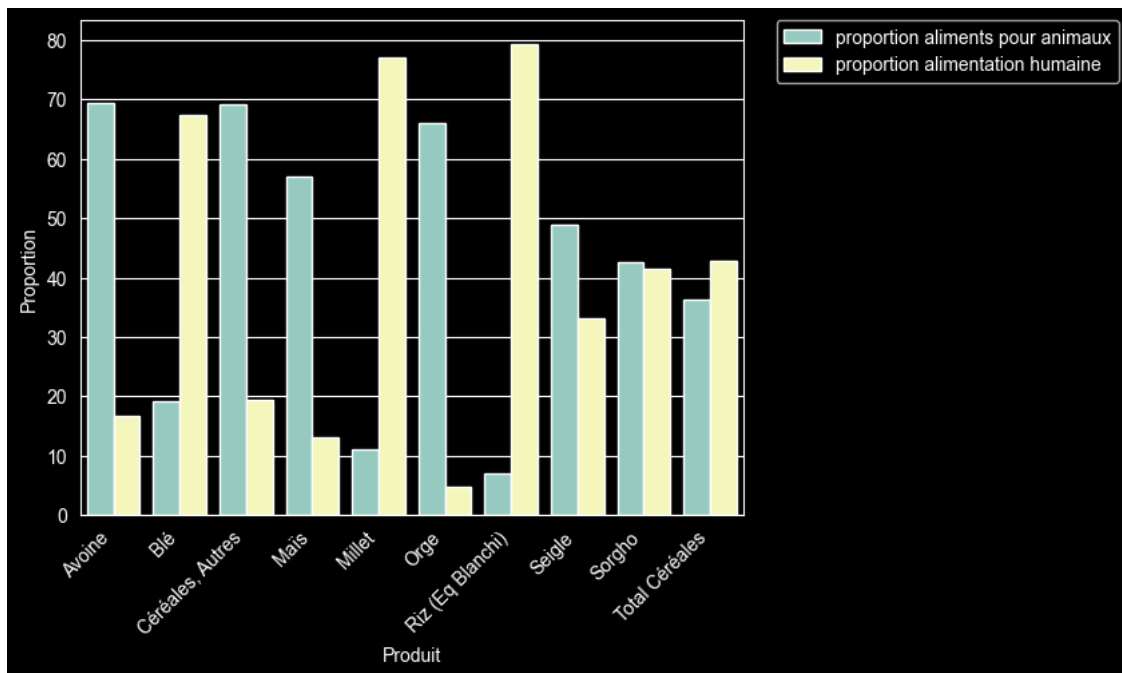
```
[161]: dispo_cereales_somme_2
```

```
[161]:
```

	Produit	Type	Proportion
0	Avoine	proportion aliments pour animaux	69.427949
1	Blé	proportion aliments pour animaux	19.082911

2	Céréales, Autres	proportion aliments pour animaux	69.255958
3	Maïs	proportion aliments pour animaux	57.137118
4	Millet	proportion aliments pour animaux	11.052790
5	Orge	proportion aliments pour animaux	65.977399
6	Riz (Eq Blanchi)	proportion aliments pour animaux	7.062667
7	Seigle	proportion aliments pour animaux	48.886340
8	Sorgho	proportion aliments pour animaux	42.598348
9	Total Céréales	proportion aliments pour animaux	36.291457
10	Avoine	proportion alimentation humaine	16.674499
11	Blé	proportion alimentation humaine	67.376799
12	Céréales, Autres	proportion alimentation humaine	19.370566
13	Maïs	proportion alimentation humaine	13.097314
14	Millet	proportion alimentation humaine	77.028518
15	Orge	proportion alimentation humaine	4.837688
16	Riz (Eq Blanchi)	proportion alimentation humaine	79.319088
17	Seigle	proportion alimentation humaine	33.210599
18	Sorgho	proportion alimentation humaine	41.473634
19	Total Céréales	proportion alimentation humaine	42.750745

```
[205]: sns.barplot(data=dispo_cereales_somme_2, x='Produit', y='Proportion',
hue='Type')
plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0.)
plt.xticks(rotation=45, ha='right')
plt.show()
```



3.6 - Pays avec la proportion de personnes sous-alimentée la plus forte en 2017

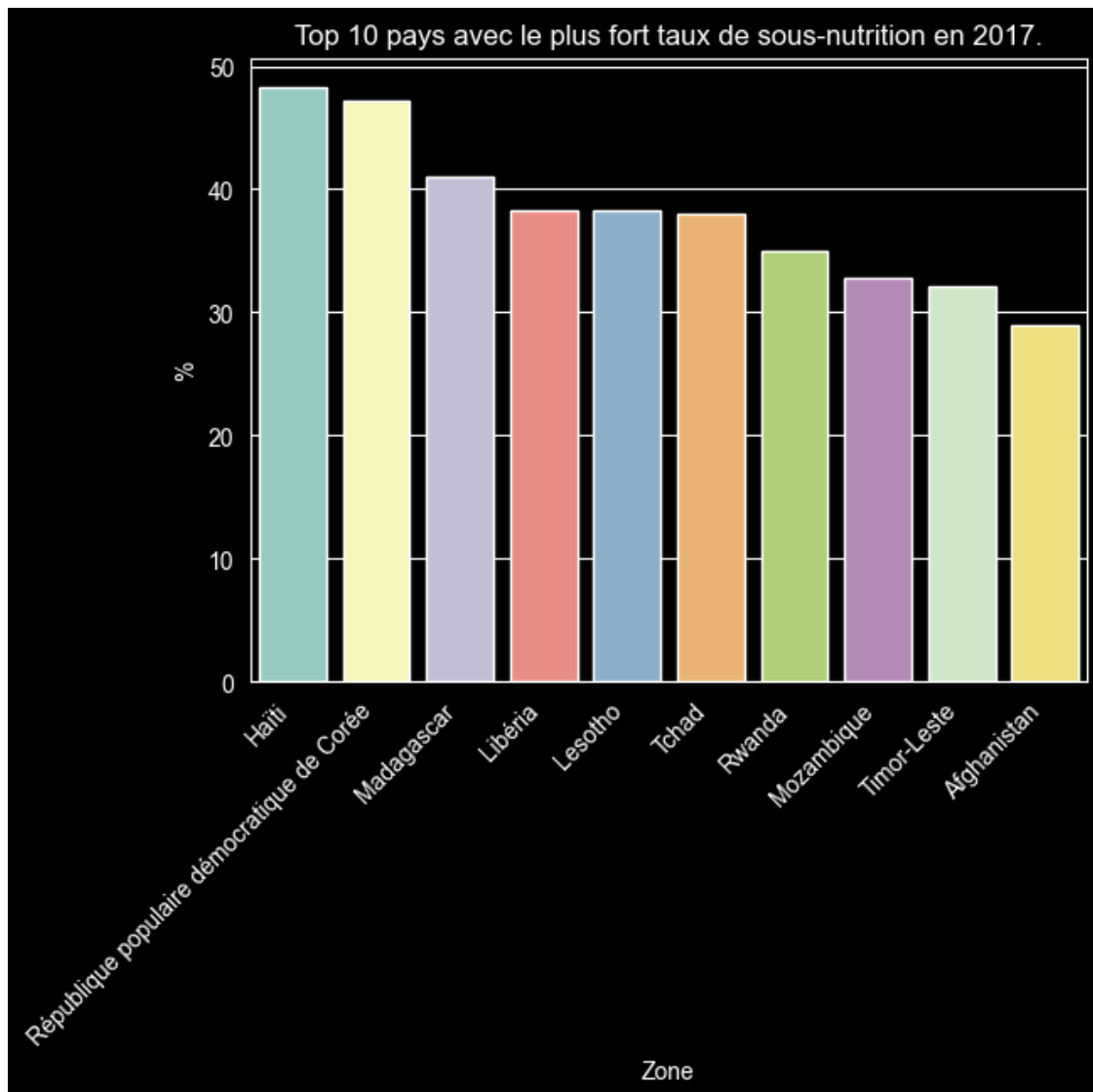
```
[163]: #Création de la colonne proportion par pays
proportion_sous_nutrition_2017['Proportion en %'] =
    ↪proportion_sous_nutrition_2017['sous_nutrition']*100 /
    ↪proportion_sous_nutrition_2017['Population']

[164]: #affichage après trie des 10 pires pays
proportion_sous_nutrition_2017.sort_values('Proportion en %', ascending=False,
    ↪inplace=True)

#nouveau DF ne contenant que les 10 pires pays
top_10_pays_sous_nutrition = proportion_sous_nutrition_2017.head(10)

plt.xticks(rotation=45, ha='right')
plt.ylabel('%')
plt.title("Top 10 pays avec le plus fort taux de sous-nutrition en 2017.")
sns.barplot(data=top_10_pays_sous_nutrition, x='Zone', y='Proportion en %',
    ↪hue='Zone')
```

[164]: <Axes: title={'center': 'Top 10 pays avec le plus fort taux de sous-nutrition en 2017.'}, xlabel='Zone', ylabel='%>



3.7 - Pays qui ont le plus bénéficié d'aide alimentaire depuis 2013

```
[165]: #calcul du total de l'aide alimentaire par pays
total_aide_pays = aide_alimentaire.groupby('Zone')['Valeur'].sum().reset_index()

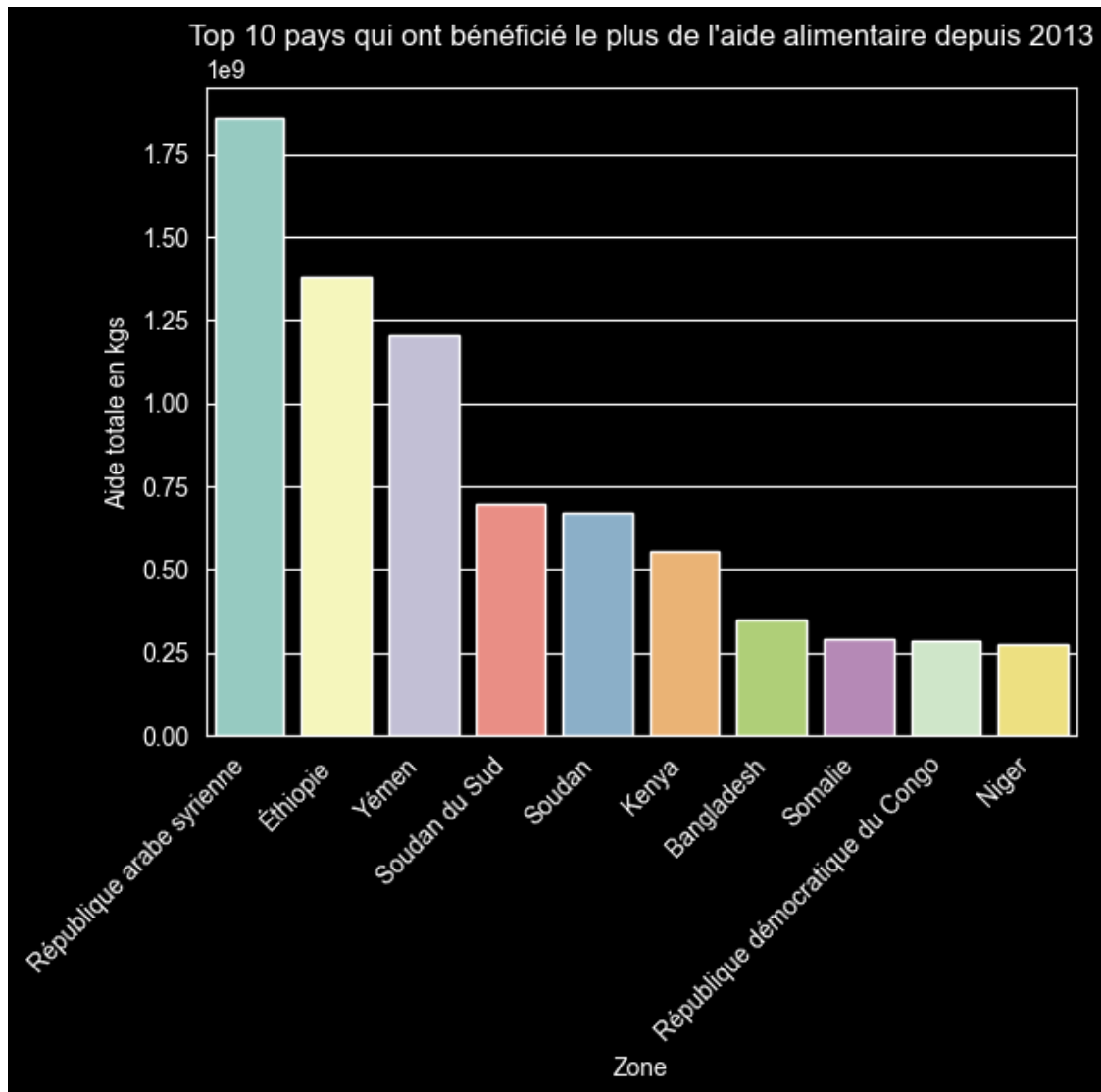
[166]: #affichage après tri des 10 pays qui ont bénéficié le plus de l'aide alimentaire
total_aide_pays.sort_values(by='Valeur', ascending=False, inplace=True)

#création d'un DF avec les 10 pays
top_10_pays_aide = total_aide_pays.head(10)

plt.xticks(rotation=45, ha='right')
plt.ylabel('Aide totale en kgs')
```

```
plt.title("Top 10 pays qui ont bénéficié le plus de l'aide alimentaire depuis 2013")
sns.barplot(data=top_10_pays_aide, x='Zone', y='Valeur', hue='Zone')
```

[166]: <Axes: title={'center': "Top 10 pays qui ont bénéficié le plus de l'aide alimentaire depuis 2013"}, xlabel='Zone', ylabel='Aide totale en kgs'>



3.8 - Evolution des 5 pays qui ont le plus bénéficiés de l'aide alimentaire entre 2013 et 2016

[167]: #Création d'un dataframe avec la zone, l'année et l'aide alimentaire puis
 ↳groupby sur zone et année (il y a une entrée pour chaque produit dans chaque
 ↳pays chaque année, il faut agréger cela)
 aide_alimentaire_evolution = aide_alimentaire[['Zone', 'Année', 'Valeur']]

```
aide_alimentaire_evolution = aide_alimentaire_evolution.groupby(['Zone',  
↪ 'Année']).sum().reset_index()  
aide_alimentaire_evolution
```

```
[167]:
```

	Zone	Année	Valeur
0	Afghanistan	2013	128238000
1	Afghanistan	2014	57214000
2	Algérie	2013	35234000
3	Algérie	2014	18980000
4	Algérie	2015	17424000
..	...	...	...
223	Égypte	2013	1122000
224	Équateur	2013	1362000
225	Éthiopie	2013	591404000
226	Éthiopie	2014	586624000
227	Éthiopie	2015	203266000

[228 rows x 3 columns]

```
[168]: #Création d'une liste contenant les 5 pays qui ont le plus bénéficié de l'aide  
↪ alimentaire  
liste_top_5 = total_aide_pays['Zone'].head(5).tolist()  
print(liste_top_5)
```

['République arabe syrienne', 'Éthiopie', 'Yémen', 'Soudan du Sud', 'Soudan']

```
[169]: #On filtre sur le dataframe avec notre liste  
mask_top5_aide = aide_alimentaire_evolution['Zone'].isin(liste_top_5)  
evolution_aide_top5 = aide_alimentaire_evolution.loc[mask_top5_aide]  
evolution_aide_top5
```

```
[169]:
```

	Zone	Année	Valeur
157	République arabe syrienne	2013	563566000
158	République arabe syrienne	2014	651870000
159	République arabe syrienne	2015	524949000
160	République arabe syrienne	2016	118558000
189	Soudan	2013	330230000
190	Soudan	2014	321904000
191	Soudan	2015	17650000
192	Soudan du Sud	2013	196330000
193	Soudan du Sud	2014	450610000
194	Soudan du Sud	2015	48308000
214	Yémen	2013	264764000
215	Yémen	2014	103840000
216	Yémen	2015	372306000
217	Yémen	2016	465574000
225	Éthiopie	2013	591404000
226	Éthiopie	2014	586624000

```
[170]: #changement des valeurs en numérique pour pouvoir les diviser en millions de
↳kgs pour plus de clarté
evolution_aide_top5['Valeur'] = pd.to_numeric(evolution_aide_top5['Valeur'],
↳errors='coerce')
```

C:\Users\MonCompte\AppData\Local\Temp\ipykernel_34980\4127911073.py:2:

SettingWithCopyWarning:

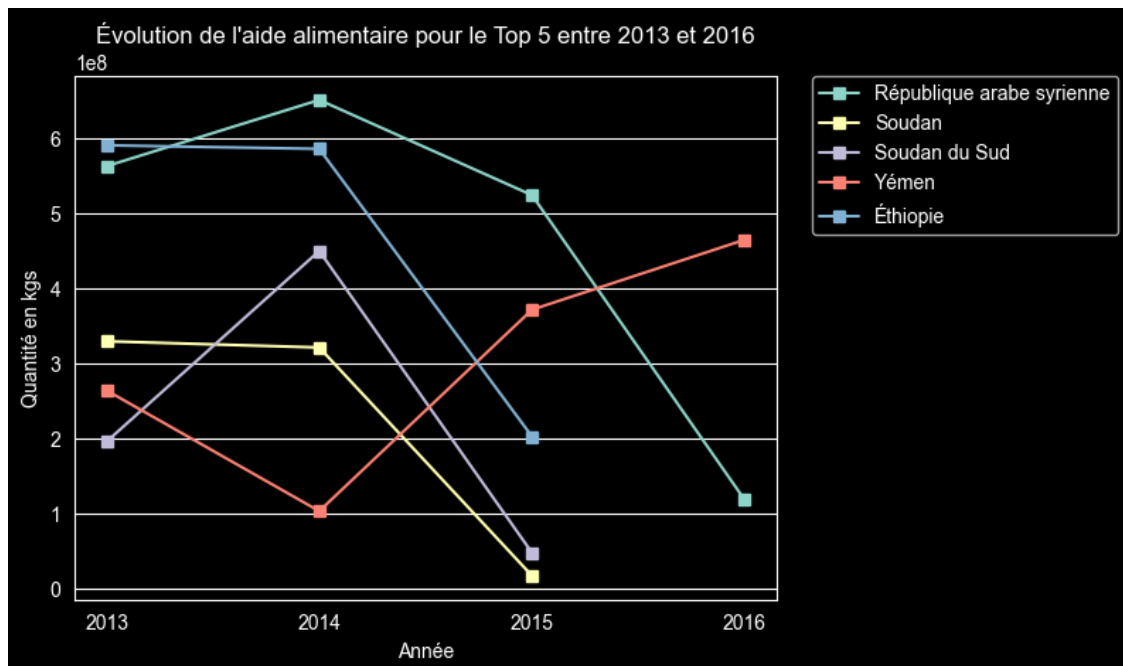
A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
evolution_aide_top5['Valeur'] = pd.to_numeric(evolution_aide_top5['Valeur'],
errors='coerce')
```

```
[206]: # Affichage des pays avec l'aide alimentaire par année",
evolution_aide_top5.pivot(index='Année', columns='Zone', values='Valeur').
↳plot(kind='line', marker='s')
plt.xticks([2013, 2014, 2015, 2016])
plt.title("Évolution de l'aide alimentaire pour le Top 5 entre 2013 et 2016")
plt.ylabel("Quantité en kgs")
plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0.)
plt.grid(axis='x')
plt.show()
```



3.9 - Pays avec le moins de disponibilité par habitant

```
[172]: #Calcul de la disponibilité en kcal par personne par jour par pays
# On crée un DF avec la somme de dispo par pays
dispo_par_pays = dispo_alimentaire.groupby('Zone')['Disponibilité alimentaire_
↳(Kcal/personne/jour)'].sum().reset_index()
dispo_par_pays
```

```
[172]:
```

	Zone	Disponibilité alimentaire (Kcal/personne/jour)
0	Afghanistan	2087.0
1	Afrique du Sud	3020.0
2	Albanie	3188.0
3	Algérie	3293.0
4	Allemagne	3503.0
..	...	...
169	Émirats arabes unis	3275.0
170	Équateur	2346.0
171	États-Unis d'Amérique	3682.0
172	Éthiopie	2129.0
173	Îles Salomon	2383.0

[174 rows x 2 columns]

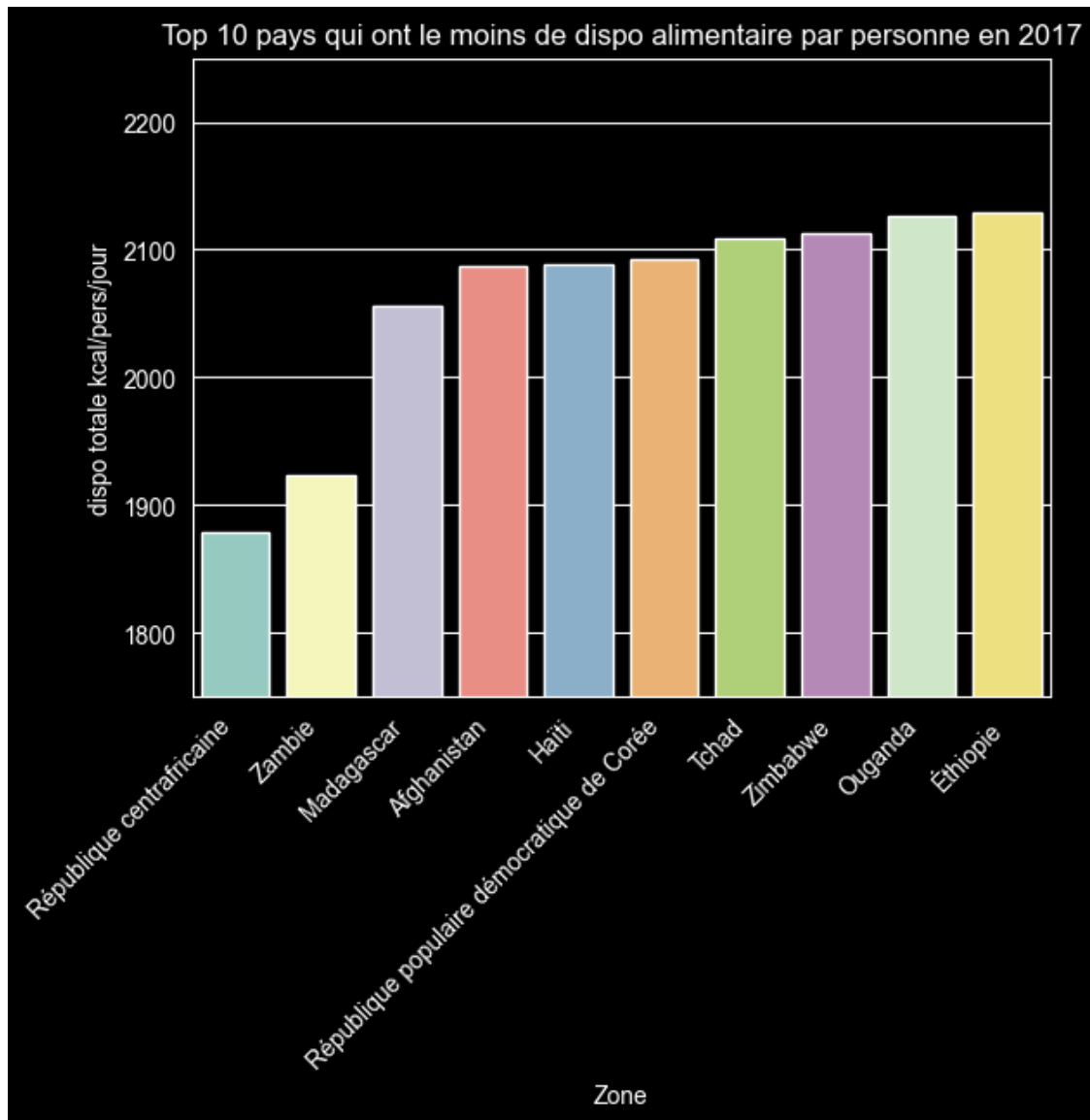
```
[172]:
```

```
[173]: #Affichage des 10 pays qui ont le moins de dispo alimentaire par personne
#On trie avec ascending True pour avoir les 10 pires en 1er
dispo_par_pays_asc = dispo_par_pays.sort_values(by='Disponibilité alimentaire_
↳(Kcal/personne/jour)', ascending=True)

top_10_pays_dispo_asc = dispo_par_pays_asc.head(10)

plt.xticks(rotation=45, ha='right')
plt.ylim(1750, 2250)
plt.ylabel('dispo totale kcal/pers/jour')
plt.title("Top 10 pays qui ont le moins de dispo alimentaire par personne en_
↳2017")
sns.barplot(data=top_10_pays_dispo_asc, x='Zone', y='Disponibilité alimentaire_
↳(Kcal/personne/jour)', hue='Zone')
```

```
[173]: <Axes: title={'center': 'Top 10 pays qui ont le moins de dispo alimentaire par
personne en 2017'}, xlabel='Zone', ylabel='dispo totale kcal/pers/jour'>
```



3.10 - Pays avec le plus de disponibilité par habitant

```
[174]: #Affichage des 10 pays qui ont le plus de dispo alimentaire par personne
#On trie avec ascending False pour avoir les 10 meilleurs en 1er
dispo_par_pays_desc = dispo_par_pays.sort_values(by='Disponibilité alimentaire_
↳(Kcal/personne/jour)', ascending=False)
dispo_par_pays_desc.head(10)

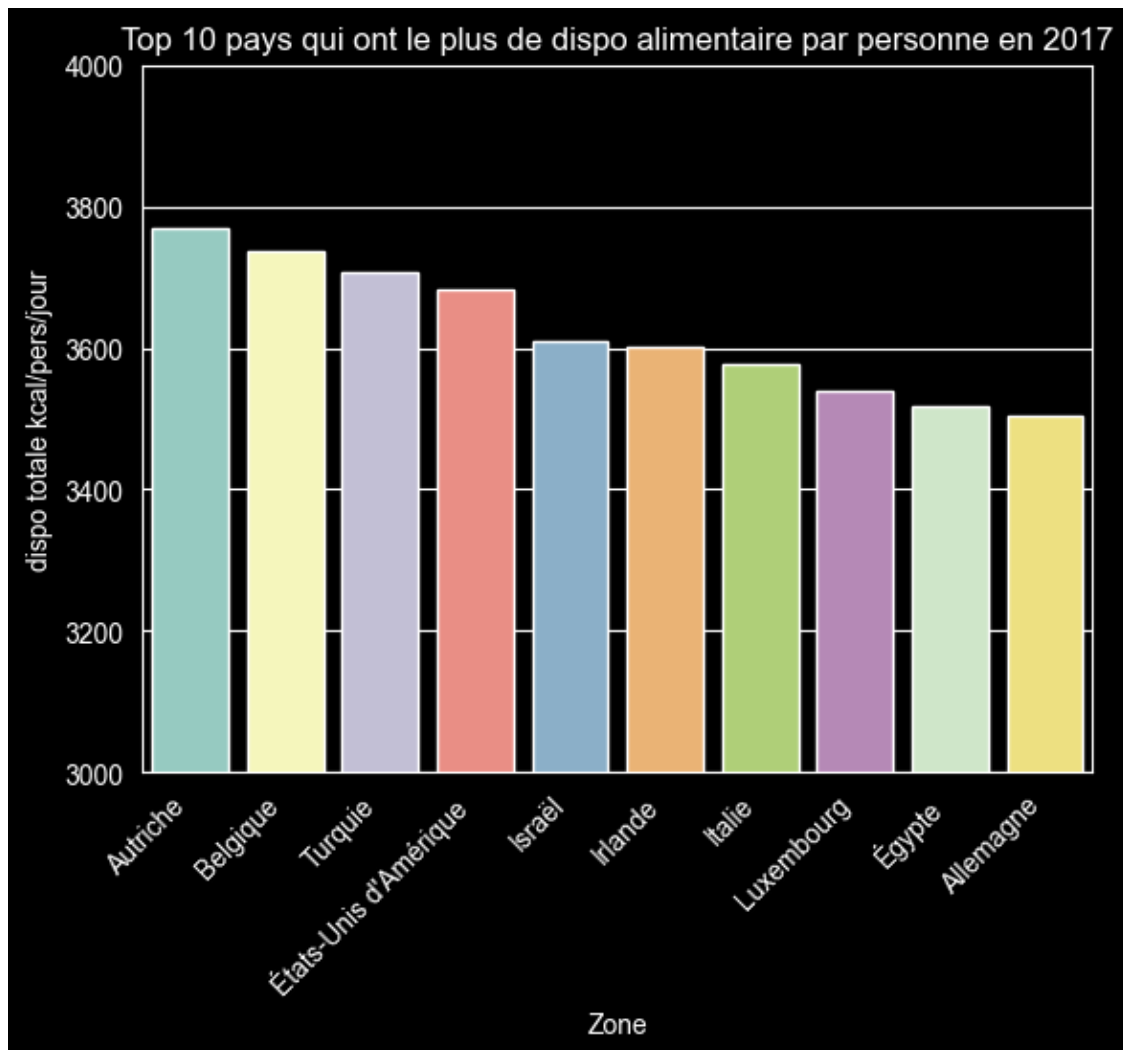
top_10_pays_dispo_desc = dispo_par_pays_desc.head(10).head(10)

plt.xticks(rotation=45, ha='right')
plt.ylim(3000, 4000)
```



```
plt.ylabel('dispo totale kcal/pers/jour')
plt.title("Top 10 pays qui ont le plus de dispo alimentaire par personne en 2017")
sns.barplot(data=top_10_pays_dispo_desc, x='Zone', y='Disponibilité alimentaire (Kcal/personne/jour)', hue='Zone')
```

[174]: <Axes: title={'center': 'Top 10 pays qui ont le plus de dispo alimentaire par personne en 2017'}, xlabel='Zone', ylabel='dispo totale kcal/pers/jour'>



3.11 - Exemple de la Thaïlande pour le Manioc

```
[175]: #création d'un dataframe avec uniquement la Thaïlande
df_thaïlande = dispo_alimentaire.loc[dispo_alimentaire['Zone'] == 'Thaïlande']
df_thaïlande
```

[175]:

	Zone	Produit	Origine	Aliments pour animaux \
13759	Thaïlande	Abats Comestible	animale	0.0
13760	Thaïlande	Agrumes, Autres	vegetale	0.0
13761	Thaïlande	Alcool, non Comestible	vegetale	0.0
13762	Thaïlande	Aliments pour enfants	vegetale	0.0
13763	Thaïlande	Ananas	vegetale	0.0
...	...	...	...	...
13849	Thaïlande	Viande de Suides	animale	0.0
13850	Thaïlande	Viande de Volailles	animale	0.0
13851	Thaïlande	Viande, Autre	animale	0.0
13852	Thaïlande	Vin	vegetale	0.0
13853	Thaïlande	Épices, Autres	vegetale	0.0

	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour) \
13759	0.0	3.0
13760	0.0	0.0
13761	358000000.0	0.0
13762	0.0	2.0
13763	0.0	10.0
...	...	...
13849	0.0	124.0
13850	0.0	52.0
13851	0.0	0.0
13852	0.0	0.0
13853	0.0	16.0

	Disponibilité alimentaire en quantité (kg/personne/an) \
13759	1.11
13760	0.09
13761	0.00
13762	0.18
13763	10.02
...	...
13849	13.00
13850	13.69
13851	0.03
13852	0.12
13853	1.70

	Disponibilité de matière grasse en quantité (g/personne/jour) \
13759	0.09
13760	0.00
13761	0.00
13762	0.01
13763	0.04
...	...
13849	11.83

13850	3.62
13851	0.01
13852	0.00
13853	0.30

Disponibilité de protéines en quantité (g/personne/jour) \	
13759	0.56
13760	0.00
13761	0.00
13762	0.08
13763	0.08
...	...
13849	3.92
13850	4.49
13851	0.02
13852	0.00
13853	0.43

Disponibilité intérieure ... Importations - Quantité Nourriture \				
13759	74000000.0	...	33000000.0	75000000.0
13760	8000000.0	...	2000000.0	6000000.0
13761	358000000.0	...	21000000.0	0.0
13762	12000000.0	...	19000000.0	12000000.0
13763	782000000.0	...	9000000.0	671000000.0
...	...	...	...	...
13849	871000000.0	...	1000000.0	871000000.0
13850	945000000.0	...	11000000.0	917000000.0
13851	-92000000.0	...	4000000.0	2000000.0
13852	8000000.0	...	16000000.0	8000000.0
13853	114000000.0	...	13000000.0	114000000.0

Pertes Production Semences Traitement Variation de stock \					
13759	0.0	4.500000e+07	0.0	0.0	0.0
13760	0.0	1.200000e+07	0.0	2000000.0	0.0
13761	0.0	4.470000e+08	0.0	0.0	0.0
13762	0.0	0.000000e+00	0.0	0.0	0.0
13763	110000000.0	2.209000e+09	0.0	0.0	13000000.0
...	...	...	...	...	...
13849	0.0	8.910000e+08	0.0	0.0	0.0
13850	280000000.0	1.470000e+09	0.0	0.0	0.0
13851	0.0	0.000000e+00	0.0	0.0	0.0
13852	0.0	0.000000e+00	0.0	0.0	0.0
13853	0.0	1.430000e+08	0.0	0.0	0.0

Année Population dispo_kcal par jour			
13759	2017	69209810.0	2.076294e+08
13760	2017	69209810.0	0.000000e+00

```

13761    2017    69209810.0      0.000000e+00
13762    2017    69209810.0      1.384196e+08
13763    2017    69209810.0      6.920981e+08
...
13849    2017    69209810.0      8.582016e+09
13850    2017    69209810.0      3.598910e+09
13851    2017    69209810.0      0.000000e+00
13852    2017    69209810.0      0.000000e+00
13853    2017    69209810.0      1.107357e+09

```

[95 rows x 21 columns]

```

[176]: #Calcul de la sous nutrition en Thaïlande
sous_nutrition_thai = proportion_sous_nutrition_2017.
↳ loc[proportion_sous_nutrition_2017['Zone'] == 'Thaïlande']
sous_nutrition_thai

```

```

[176]:      Zone  Année  sous_nutrition  Population  Proportion en %
185  Thaïlande   2017      6200000.0   69209810.0      8.958268

```

```

[177]: # On calcule la proportion exportée en fonction de la production
# Regarder les exportations et la production de Manioc
# On crée un DF avec seulement la thaïlande et le manioc
df_thaïlande_manioc = df_thaïlande.loc[df_thaïlande['Produit']=='Manioc',
↳ ['Zone', 'Produit', 'Disponibilité alimentaire en quantité (kg/personne/
↳ an)', 'Disponibilité alimentaire (Kcal/personne/jour)', 'Exportations -
↳ Quantité', 'Production']]

```

```

[178]: df_thaïlande_manioc

```

```

[178]:      Zone Produit \
13809  Thaïlande  Manioc

      Disponibilité alimentaire en quantité (kg/personne/an) \
13809                                                         13.0

      Disponibilité alimentaire (Kcal/personne/jour) \
13809                                                         40.0

      Exportations - Quantité      Production
13809      2.521400e+10  3.022800e+10

```

```

[179]: #Création d'une colonne proportion exportation sur production
df_thaïlande_manioc['proportion exportation sur production'] =
↳ df_thaïlande_manioc['Exportations - Quantité'] * 100 /
↳ df_thaïlande_manioc['Production']

```

```
[180]: print(f"La proportion de Manioc exporté sur la production en Thaïlande est de {round(df_thaïlande_manioc['proportion exportation sur production'].sum(), 2)}%.")
```

La proportion de Manioc exporté sur la production en Thaïlande est de 83.41%.

Etape 6 - Analyse complémentaires

```
[181]: #ajouter en dessous toutes les analyses complémentaires
```

Comparaison de l'aide alimentaire par rapport à la sous nutrition chaque année

```
[182]: aide_alimentaire['Année'].unique()
```

```
[182]: array([2013, 2014, 2015, 2016])
```

```
[183]: #On crée un DF pour avoir la somme par et par pays de l'aide alimentaire
somme_aide_par_an_et_pays = aide_alimentaire.groupby(by=['Zone', 'Année'])['Valeur'].sum().reset_index()
somme_aide_par_an_et_pays.head()
```

```
[183]:
```

	Zone	Année	Valeur
0	Afghanistan	2013	128238000
1	Afghanistan	2014	57214000
2	Algérie	2013	35234000
3	Algérie	2014	18980000
4	Algérie	2015	17424000

```
[184]: # On vérifie que les deux colonnes 'Zone' ont bien les mêmes données : on fait la différence entre les 2 séries en les transformant en set car plus rapide.
manquants_sous_nutrition = list(set(somme_aide_par_an_et_pays['Zone'] ) - set(sous_nutrition['Zone']))
manquants_aide = list(set(sous_nutrition['Zone'] ) - set(somme_aide_par_an_et_pays['Zone']))
print(f"Pays dans aide mais pas dans sous_nutrition: {len(manquants_sous_nutrition)}")
print(f"Pays dans sous_nutrition mais pas dans aide: {len(manquants_aide)}")
```

Pays dans aide mais pas dans sous_nutrition: 1

Pays dans sous_nutrition mais pas dans aide: 128

```
[185]: print(manquants_sous_nutrition)
print(manquants_aide)
```

```
['Swaziland']
['Samoa américaines', 'Bélarus', 'Chypre', 'Suriname', 'Slovaquie', "États-Unis d'Amérique", 'Suède', 'Hongrie', 'Thaïlande', 'Saint-Vincent-et-les Grenadines', 'Cabo Verde', 'Islande', 'Azerbaïdjan', 'Chili', 'Malaisie', 'Nouvelle-Calédonie', 'Oman', 'Albanie', 'Grèce', 'Bulgarie', "Royaume-Uni de Grande-Bretagne et d'Irlande du Nord", 'Maurice', 'Italie', 'Luxembourg', 'Costa Rica',
```

'Espagne', 'Dominique', 'Roumanie', 'Mongolie', 'Ouzbékistan', 'Estonie',
 'Tchéquie', 'Turquie', 'Maldives', 'Érythrée', 'Tonga', 'Malte', 'Barbade',
 'Allemagne', 'Jamaïque', 'Gabon', 'Paraguay', 'Brésil', 'Grenade', 'Monténégro',
 'Croatie', 'Fédération de Russie', 'Serbie', 'Îles Marshall', 'Samoa',
 'Nigéria', 'Papouasie-Nouvelle-Guinée', 'Inde', 'Mexique', 'Fidji', 'Guinée
 équatoriale', 'Japon', 'République de Moldova', 'Koweït', 'Chine - RAS de Hong-
 Kong', 'Pays-Bas', 'Qatar', 'Argentine', 'Eswatini', 'Polynésie française',
 'Singapour', 'Palaos', 'Îles Cook', 'Saint-Kitts-et-Nevis', 'Bahamas', 'Israël',
 'Portugal', 'Porto Rico', 'Antigua-et-Barbuda', 'Suisse', 'Andorre',
 'Australie', 'Nioué', 'Uruguay', 'Maroc', 'Pérou', 'Tunisie', 'Finlande',
 'Namibie', 'Seychelles', 'Belize', 'Bosnie-Herzégovine', 'Botswana', 'Bahreïn',
 'Irlande', 'Slovénie', 'Tuvalu', 'Arménie', 'Nouvelle-Zélande', 'Danemark',
 'Kiribati', 'Groenland', 'Îles Salomon', 'République de Corée', 'France',
 'Kazakhstan', 'Venezuela (République bolivarienne du)', 'Chine, Taiwan Province
 de', 'Viet Nam', 'Chine - RAS de Macao', 'Belgique', 'Afrique du Sud',
 'Bermudes', 'Brunéi Darussalam', 'Ukraine', 'Turkménistan', 'Macédoine du Nord',
 'Nauru', 'Autriche', 'Canada', 'Émirats arabes unis', 'Tokélaou', 'Norvège',
 'Micronésie (États fédérés de)', 'Lettonie', 'Trinité-et-Tobago', 'Guyana',
 'Pologne', 'Indonésie', 'Lituanie', 'Sainte-Lucie', 'Arabie saoudite', 'Panama']

```
[186]: # on merge nos deux DF dans un seul en faisant une jointure à gauche sur
      ↪somme_aide_par_an_et_pays
aide_sous_nutrition = pd.merge(somme_aide_par_an_et_pays, sous_nutrition,
      ↪on=['Zone', 'Année'], how="left")
aide_sous_nutrition
```

```
[186]:
```

	Zone	Année	Valeur	sous_nutrition
0	Afghanistan	2013	128238000	8600000.0
1	Afghanistan	2014	57214000	8800000.0
2	Algérie	2013	35234000	1300000.0
3	Algérie	2014	18980000	1300000.0
4	Algérie	2015	17424000	1300000.0
..	...	...	...	...
223	Égypte	2013	1122000	4400000.0
224	Équateur	2013	1362000	2300000.0
225	Éthiopie	2013	591404000	26200000.0
226	Éthiopie	2014	586624000	24300000.0
227	Éthiopie	2015	203266000	21700000.0

[228 rows x 4 columns]

```
[187]: aide_sous_nutrition[aide_sous_nutrition['sous_nutrition'] == 0].count()
```

```
[187]: Zone          68
      Année       68
      Valeur      68
      sous_nutrition 68
```

dtype: int64

```
[188]: df = aide_sous_nutrition[aide_sous_nutrition['Valeur'] == 0]
df
```

```
[188]:      Zone  Année  Valeur  sous_nutrition
206  Tadjikistan  2015      0           0.0
```

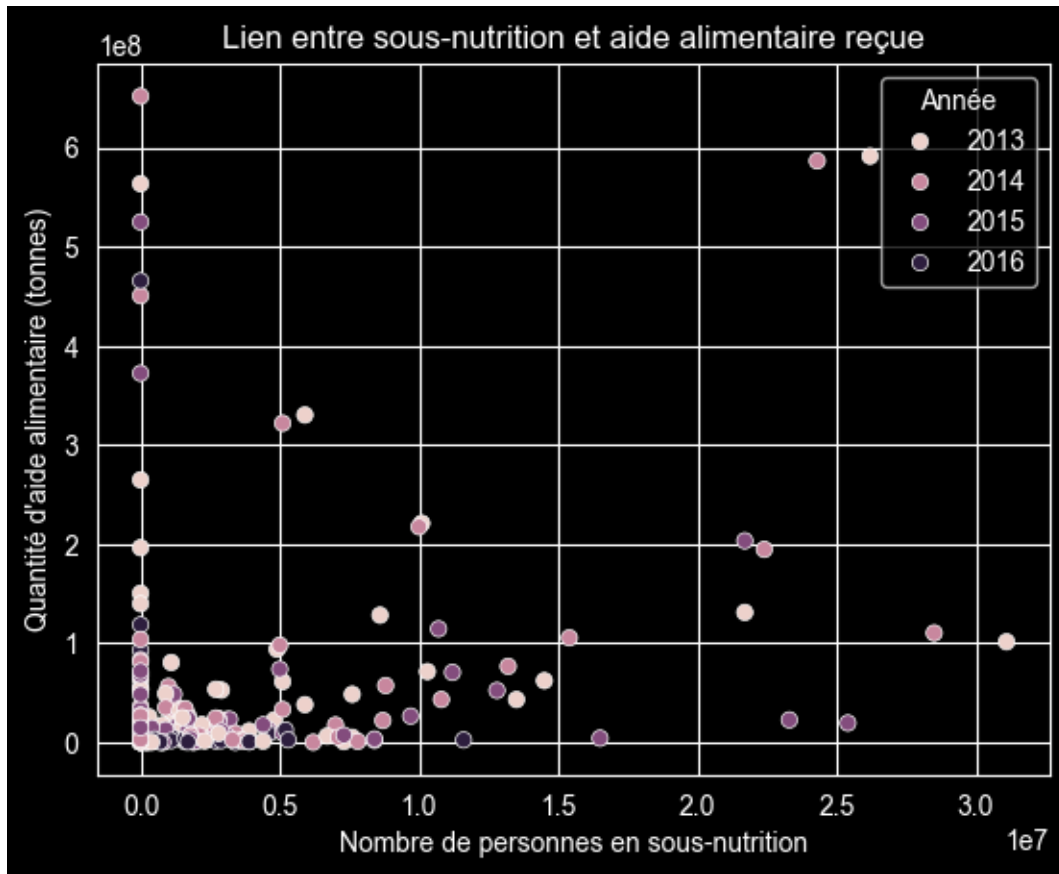
```
[189]: # On nettoie notre DF en enlevant les lignes ayant une valeur nulle sur les
      ↪ colonnes aides ou sous nutrition
aide_sous_nutrition = aide_sous_nutrition.dropna(subset=["Valeur",
      ↪ "sous_nutrition"])
```

```
[190]: # Graph en nuage de points entre les deux variables sur toutes les années

sns.scatterplot(data=aide_sous_nutrition, x="sous_nutrition", y="Valeur",
      ↪ hue='Année')
plt.title("Lien entre sous-nutrition et aide alimentaire reçue")
plt.xlabel("Nombre de personnes en sous-nutrition")
plt.ylabel("Quantité d'aide alimentaire (tonnes)")
plt.show()

#Ajout du R² et de la p-value
import scipy.stats as st

print(f"Coefficient de corrélation : {st.
      ↪ pearsonr(aide_sous_nutrition["sous_nutrition"],aide_sous_nutrition["Valeur"])[0]:
      ↪ .3f}")
print(f"P-value : {st.
      ↪ pearsonr(aide_sous_nutrition["sous_nutrition"],aide_sous_nutrition["Valeur"])[1]:
      ↪ .3e}")
```



Coefficient de corrélation : 0.267

P-value : 4.915e-05

[190]:

[191]: *#On va produire une série de nuages pour chaque année et observer si certaines*
→ sont plus corrélées que d'autres

liste_annees = aide_sous_nutrition['Année'].unique() # on fait une liste avec
→ les années pour faire une boucle for dessus et créer un graph par an

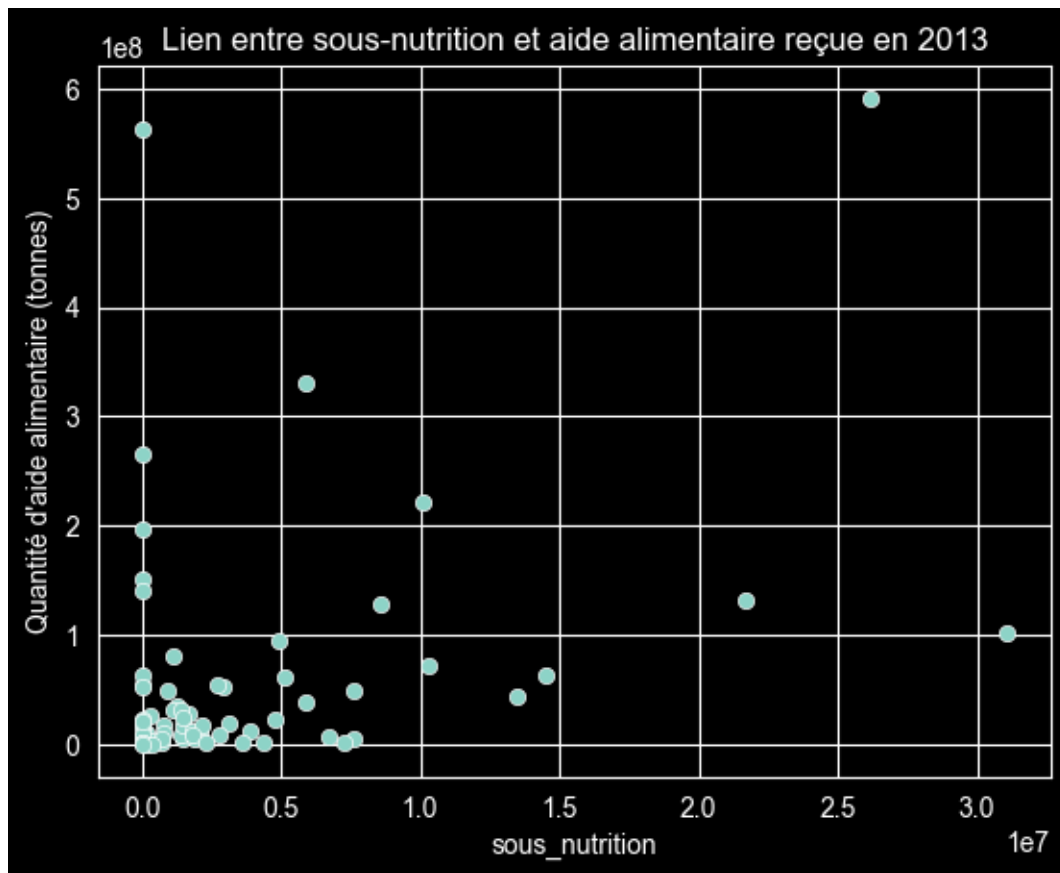
```
for year in liste_annees:
    df = aide_sous_nutrition.loc[aide_sous_nutrition["Année"] == year]
    sns.scatterplot(data=df, x="sous_nutrition", y="Valeur")
    plt.title("Lien entre sous-nutrition et aide alimentaire reçue en {0}".
    → format(year))
    plt.ylabel("Quantité d'aide alimentaire (tonnes)")
    print(f"Coefficient de corrélation : {st.
    → pearsonr(df["sous_nutrition"],df["Valeur"])[0]:.3f}")
    print(f"P-value : {st.pearsonr(df["sous_nutrition"],df["Valeur"])[1]:.3e}")
```



```
plt.show()
```

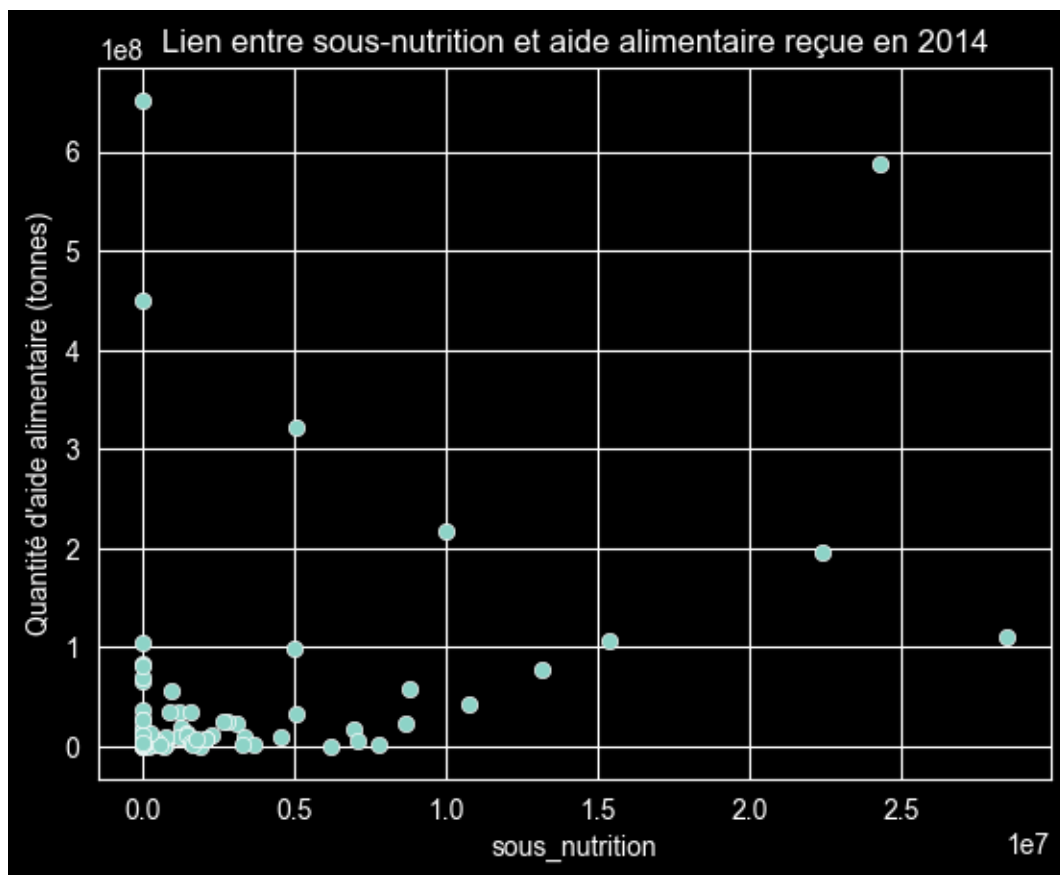
Coefficient de corrélation : 0.380

P-value : 1.083e-03



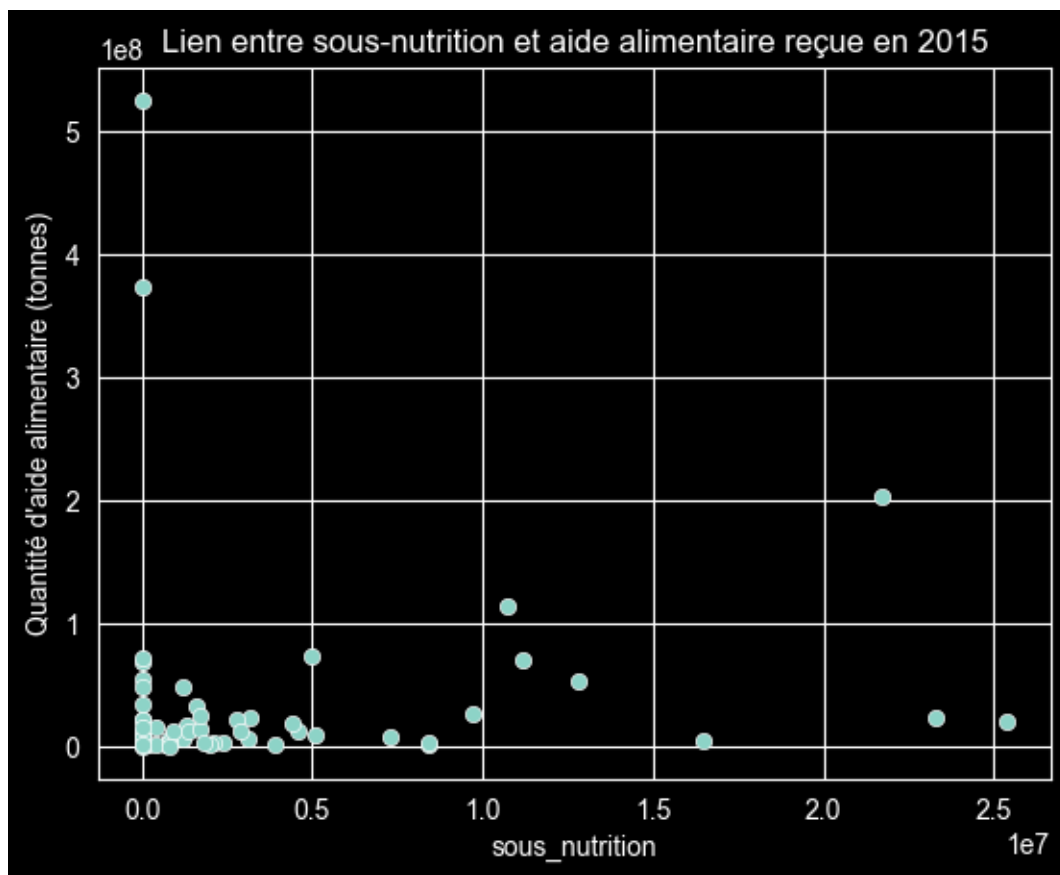
Coefficient de corrélation : 0.358

P-value : 2.529e-03



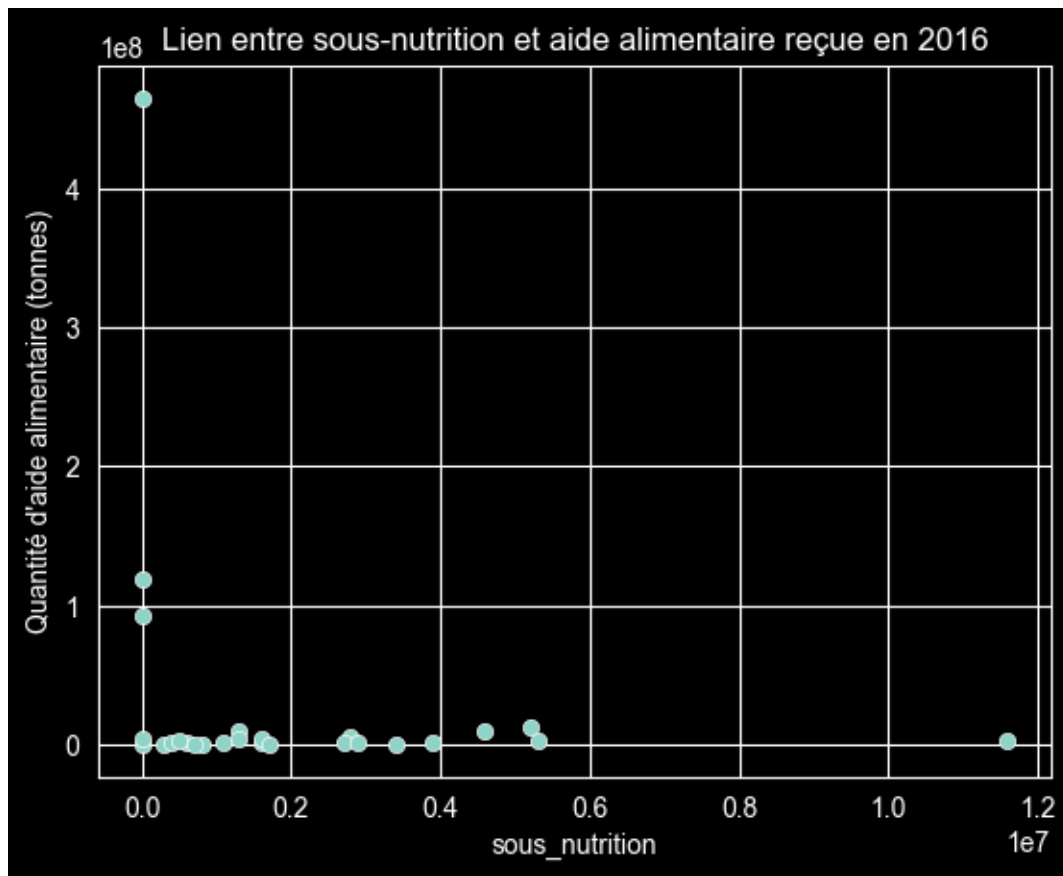
Coefficient de corrélation : 0.058

P-value : 6.581e-01



Coefficient de corrélation : -0.227

P-value : 2.656e-01



Analyse des pertes

```
[192]: # Dataframe avec le total de pertes par pays
dispo_alimentaire_pertes = dispo_alimentaire.groupby(by='Zone')['Pertes'].sum().
    ↪reset_index()
dispo_alimentaire_pertes
```

```
[192]:
```

	Zone	Pertes
0	Afghanistan	1.135000e+09
1	Afrique du Sud	2.193000e+09
2	Albanie	2.760000e+08
3	Algérie	3.753000e+09
4	Allemagne	3.781000e+09
..	...	...
169	Émirats arabes unis	7.050000e+08
170	Équateur	7.070000e+08
171	États-Unis d'Amérique	7.162000e+09
172	Éthiopie	2.256000e+09
173	Îles Salomon	6.000000e+06

[174 rows x 2 columns]

```
[193]: #Je veux merge sous nutrition dans dispo_alimentaire_pertes, je vais donc faire
↳ la vérification habituelle de mes colonnes de jointure
```

```
[194]: # On vérifie que les deux colonnes 'Zone' ont bien les mêmes données :
manquants_sous_nutrition_2017 = list(set(dispo_alimentaire_pertes['Zone'] ) -
↳ set(sous_nutrition_2017['Zone']))
manquants_pertes = list(set(sous_nutrition_2017['Zone'] ) -
↳ set(dispo_alimentaire_pertes['Zone']))
print(f"Pays dans aide mais pas dans sous_nutrition:
↳ {len(manquants_sous_nutrition_2017)}")
print(f"Pays dans sous_nutrition mais pas dans aide: {len(manquants_pertes)}")
```

Pays dans aide mais pas dans sous_nutrition: 2
Pays dans sous_nutrition mais pas dans aide: 31

```
[195]: sous_nutrition_2017.loc[sous_nutrition_2017['Zone'] == "Royaume-Uni de
↳ Grande-Bretagne et d'Irlande du Nord", 'Zone'] = 'Royaume-Uni'
sous_nutrition_2017.loc[sous_nutrition_2017['Zone'] == "Tchéquie", 'Zone'] =
↳ 'Tchéquie (la)'
```

```
[196]: # On revérifie que les deux colonnes 'Zone' ont bien les mêmes données :
manquants_sous_nutrition_2017 = list(set(dispo_alimentaire_pertes['Zone'] ) -
↳ set(sous_nutrition_2017['Zone']))
print(f"Pays dans aide mais pas dans sous_nutrition:
↳ {len(manquants_sous_nutrition_2017)}")
```

Pays dans aide mais pas dans sous_nutrition: 0

```
[197]: # Je peux maintenant faire ma jointure
perte_vs_sous_nutrition = pd.merge(dispo_alimentaire_pertes,
↳ sous_nutrition_2017, on='Zone', how='left')
perte_vs_sous_nutrition.info()
perte_vs_sous_nutrition.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 174 entries, 0 to 173
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Zone             174 non-null   object
1   Pertes           174 non-null   float64
2   Année            174 non-null   int64
3   sous_nutrition   174 non-null   float64
dtypes: float64(2), int64(1), object(1)
memory usage: 5.6+ KB
```

```
[197]:
```

	Zone	Pertes	Année	sous_nutrition
0	Afghanistan	1.135000e+09	2017	10500000.0
1	Afrique du Sud	2.193000e+09	2017	3100000.0
2	Albanie	2.760000e+08	2017	100000.0
3	Algérie	3.753000e+09	2017	1300000.0
4	Allemagne	3.781000e+09	2017	0.0

```
[198]: perte_vs_sous_nutrition =
↳perte_vs_sous_nutrition[perte_vs_sous_nutrition['sous_nutrition'] != 0]
perte_vs_sous_nutrition
```

```
[198]:
```

	Zone	Pertes	Année	sous_nutrition
0	Afghanistan	1.135000e+09	2017	10500000.0
1	Afrique du Sud	2.193000e+09	2017	3100000.0
2	Albanie	2.760000e+08	2017	100000.0
3	Algérie	3.753000e+09	2017	1300000.0
5	Angola	4.799000e+09	2017	5800000.0
..	...	...	...	...
164	Viet Nam	6.743000e+09	2017	6500000.0
168	Égypte	7.608000e+09	2017	4600000.0
169	Émirats arabes unis	7.050000e+08	2017	300000.0
170	Équateur	7.070000e+08	2017	1500000.0
172	Éthiopie	2.256000e+09	2017	21100000.0

[84 rows x 4 columns]

```
[199]: # Création d'une colonne avec la perte par personne en sous nutrition dans
↳chaque pays
perte_vs_sous_nutrition['kg pertes / personnes en sous nutrition / an'] =
↳perte_vs_sous_nutrition['Pertes'] / perte_vs_sous_nutrition['sous_nutrition']
```

```
[200]: perte_vs_sous_nutrition
```

```
[200]:
```

	Zone	Pertes	Année	sous_nutrition \
0	Afghanistan	1.135000e+09	2017	10500000.0
1	Afrique du Sud	2.193000e+09	2017	3100000.0
2	Albanie	2.760000e+08	2017	100000.0
3	Algérie	3.753000e+09	2017	1300000.0
5	Angola	4.799000e+09	2017	5800000.0
..	...	...	...	...
164	Viet Nam	6.743000e+09	2017	6500000.0
168	Égypte	7.608000e+09	2017	4600000.0
169	Émirats arabes unis	7.050000e+08	2017	300000.0
170	Équateur	7.070000e+08	2017	1500000.0
172	Éthiopie	2.256000e+09	2017	21100000.0

kg pertes / personnes en sous nutrition / an

0	108.095238
1	707.419355
2	2760.000000
3	2886.923077
5	827.413793
..	...
164	1037.384615
168	1653.913043
169	2350.000000
170	471.333333
172	106.919431

[84 rows x 5 columns]

```
[201]: #Affichage des 10 pays qui ont le plus de kgs de nourriture perdu par personne
        ↪ en sous nutrition.
perte_vs_sous_nutrition = perte_vs_sous_nutrition.sort_values(by='kg pertes /
        ↪ personnes en sous nutrition / an', ascending=False)

perte_vs_sous_nutrition_top10 = perte_vs_sous_nutrition.head(10)

plt.xticks(rotation=45, ha='right')
plt.ylim()
plt.ylabel('kg / personne en sous-nutrition / an')
plt.title("Top 10 pays qui ont le plus de pertes alimentaire par personne en
        ↪ sous nutrition")
sns.barplot(data=perte_vs_sous_nutrition_top10, x='Zone', y='kg pertes /
        ↪ personnes en sous nutrition / an', hue='Zone')
```

```
[201]: <Axes: title={'center': 'Top 10 pays qui ont le plus de pertes alimentaire par
        personne en sous nutrition'}, xlabel='Zone', ylabel='kg / personne en sous-
        nutrition / an'>
```

